gemstone-rs Companion Manual

Setup, user manual, examples, cookbook, codegen, explorer, and workbench in one PDF

Generated from the Markdown sources in gemstone-rs/docs.

Contents

gemstone-rs Documentation

This directory contains the human-facing guides for `gemstone-rs`.

Start Here

Guide	Use it when
Setup Guide	You need Rust, GemStone environment variables, install commands, and first login checks.
Examples Guide	You want to know which example to run for each feature.
Feature Map	You want the Rust equivalent of `gemstone-examples plan3-map`, with crates, examples, docs, and gemstone-py references.
User Manual	You want the main Rust API surface in one place.
Cookbook	You want task-focused recipes for sessions, transactions, browser calls, codegen, explorer, and VS Code.
gemstone-py vs gemstone-rs	You want install paths, use cases, maturity, and feature differences between the Python and Rust projects.
gemstone-js vs gemstone-py	You want the TypeScript/Python comparison, maturity matrix, and gemstone-js catch-up batches.
BridgeRoot and Object Mapping	You want BridgeRoot storage, dictionary mapping, `Remote<T>` object handles, materialization profiles, and explicit Rust-to-GemStone value mapping.
Codegen Guide	You want generated Rust wrappers for GemStone classes and methods.
Codegen Profile Schema	You want project profile JSON validation for explorer and VS Code workflows.
Explorer Guide	You want the local HTTP explorer API and safety defaults.
VS Code Workbench	You want sidebar browsing, codegen preview/diff/generate, and explorer launch commands.
Screenshot Workflow	You want to refresh Explorer and Workbench images before docs or Marketplace releases.

Guide	Use it when
Performance and Safety	You want benchmark guidance, GCI loading notes, and threading rules.
Shared Core Integration	You want the plan for `gemstone-py-native` to wrap the Rust core.
Medium Article	You want an article-style explanation suitable for publishing or sharing.
Funny Introduction	You want a lighter multi-part tour of the same concepts.
Release Checklist	You want the crate, VSIX, GitHub Release, and post-release verification flow.

PDFs

Generated PDFs live in pdf/. Rebuild them after Markdown changes:

```
python3 docs/build_pdf_docs.py
```

Check docs links, index coverage, release versions, and PDF targets with:

```
make docs-link-check
make docs-index-check
python3 scripts/version_check.py
make docs-pdf-check
```

Refresh screenshots before a visual release:

```
make screenshots
```

Shortest Path

```
cargo install gemstone-rs-cli
gemstone-rs --help
```

For library use:

```
cargo add gemstone-rs
```

For local examples from a checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

For the local explorer:

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup Guide

This guide gets you from a Rust project or source checkout to a live GemStone login.

What You Need

At minimum:

- Rust stable toolchain
- access to a GemStone/S 64 stone
- GemStone client library access through `libgcirpc`
- GemStone username and password

For the full local development experience:

- a `gemstone-rs` checkout
- `cargo`, `rustfmt`, and `clippy`
- Node.js 22 when working on the VS Code extension
- a Marketplace PAT only when publishing the VSIX
- a crates.io API token only when publishing crates

Which Install Path Should I Use?

Use case	Command
Rust application dependency	`cargo add gemstone-rs`
CLI tools	`cargo install gemstone-rs-cli`
Local HTTP explorer	`cargo install gemstone-rs-explorer`
Source checkout examples	`cargo run -p gemstone-rs --example quickstart`
VS Code users	Install `unicompute.gemstone-rs-workbench` from Marketplace

Environment Variables

Most live commands use `Config::from_env()`:

```
gemstone-rs env sample
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Setting both to the same value keeps CLI, examples, explorer, and VS Code settings aligned.

`gemstone-rs env sample` prints a safe shell export template. It carries over current non-secret values such as `GS_LIB` or `GS_STONE`, comments optional values when they are absent, and prints placeholders for `GS_PASSWORD` and `GS_HOST_PASSWORD` instead of copying secrets.

Use `gemstone-rs env write` to save the same template:

```
gemstone-rs env write
gemstone-rs env write .env.gemstone-rs --force
```

It refuses to overwrite an existing file unless `--force` is passed. Use the file directly without sourcing it:

```
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
```

```
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

Optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc.dylib
```

Variable	Required	Purpose
`GS_LIB`	Usually	GemStone `lib/` directory for runtime library discovery.
`GS_LIB_PATH`	No	Full path to a specific `libgcirpc` file.
`GS_STONE`	Yes	Stone name.
`GS_STONE_NAME`	No	Stone-name alias used when `GS_STONE` is absent.
`GS_USERNAME`	Yes	GemStone login username.
`GS_PASSWORD`	Yes	GemStone login password.
`GS_HOST`	No	Remote host, defaults to local behavior.
`GS_NETLDI`	No	NetLDI service name.
`GS_GEM_SERVICE`	No	Gem service name.

First Login

With the CLI:

```
cargo install gemstone-rs-cli
gemstone-rs env sample
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs --env-file .env.gemstone-rs codegen explain --json gemstone-rs.codegen
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

`gemstone-rs doctor` prints the relevant `GS_*` values with secrets masked, loads the configured `libgcirpc`, and reports setup problems without exposing passwords. The GCI section reports the source that selected the library: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. Add `--live` when the stone should be reachable; it logs in and asserts that `3 + 4` returns `7`. Add `--json` when a script, CI job, or editor integration needs a parseable report. Failed checks include hints for missing credentials, `libgcirpc` path/loading problems, and live stone connectivity. `--strict` is useful for CI because it also fails when `GS_STONE` / `GS_STONE_NAME` or a deterministic GCI library source (`GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE`) is missing. The GCI section reports whether the selected `libgcirpc` path exists, is a file, is readable, and whether the path appears to reference arm64 or x86_64.

From a source checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example hello_gemstone
cargo run -p gemstone-rs --example quickstart
```

From a Rust application:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
    Ok(())
}
```

Development Checkout

```
git clone git@github.com:unicompute/gemstone-rs.git
cd gemstone-rs
make verify
```

`make verify` runs formatting, `cargo check`, clippy, tests, codegen freshness, the VS Code extension syntax/smoke checks, and PDF generation.

Package the VSIX locally:

```
make vscode-package
```

Verify already-published artifacts:

```
scripts/publish_verify.sh 0.2.2
```

User Manual

`gemstone-rs` is the safe Rust client API for GemStone/S over GCI. Rust code imports it as `gemstone_rs`.

Configuration

Use `Config::from_env()` for normal applications:

```
let config = gemstone_rs::Config::from_env()?;
```

Use the builder when you want explicit configuration:

```
use gemstone_rs::Config;

let config = Config::builder()
    .stone("gs64stone")
    .host("localhost")
    .netldi("netldi")
    .username("DataCurator")
    .password("swordfish")
    .build()?;
```

Sessions

`Session` logs out automatically on drop. Calling `logout()` explicitly is still useful in examples and command-line tools.

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
let value = session.eval("3 + 4")?;
```

```
assert_eq!(value, Value::SmallInt(7));
session.logout()?;
```

`Session` is deliberately not `Send` or `Sync`. Keep a session on the thread that logged it in.

Use `SessionWorker` when an application server or async runtime needs shared access to a GemStone session lane without moving `Session` across threads. The worker logs in on a dedicated thread and serializes calls onto that thread:

```
use gemstone_rs::{Config, SessionWorker, Value};

let worker = SessionWorker::start(Config::from_env()?)?;
assert_eq!(worker.eval("3 + 4")?, Value::SmallInt(7));

let printed = worker.perform_oop(gemstone_rs::Oop::from_smallint(7), "printString",
&[])?;
assert_eq!(worker.fetch_string(printed)?, "7");

worker.shutdown()?;
```

Use `SessionWorkerPool` when a web service needs several independent GemStone session lanes behind one cloneable handle:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval("3 + 4")?, Value::SmallInt(7));
assert_eq!(pool.eval("40 + 2")?, Value::SmallInt(42));
pool.shutdown()?;
```

Use the async facade when an async runtime should await worker-pool work without moving `Session` across threads:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval_async("3 + 4").await?, Value::SmallInt(7));
let printed = pool
    .perform_oop_async(gemstone_rs::Oop::from_smallint(7), "printString", &[])
    .await?;
assert_eq!(pool.fetch_string_async(printed).await?, "7");
pool.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

Use `py_native` when you are building or testing a thin PyO3 wrapper for `gemstone-py-native`. It keeps the wrapper contract plain Rust while reusing the same `Session` implementation:

```
gemstone-rs py-native smoke --dry-run
gemstone-rs py-native smoke
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility --json
gemstone-rs py-native conformance --json
gemstone-rs py-native handoff --json
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-all --json
```

```
use gemstone_rs::py_native::{PyNativeSession, PyNativeValue};

let mut session = PyNativeSession::login_from_env()?;
assert_eq!(session.eval("3 + 4")?, PyNativeValue::SmallInt(7));
session.logout()?;
```

`py-native migration --json` prints the shared-core checklist and current status, `py-native compatibility --json` prints the Python package-layer method map, and `py-native conformance --json` prints the PyO3 module/session/shim surface that downstream `gemstone-py-native` integration should preserve. `py-native handoff --json` bundles the downstream acceptance artifacts, and `py-native publish-receipt --json` records the verified TestPyPI/PyPI native wheel publish runs.

Eval and Perform

`eval` returns a marshalled `Value`:

```
let value = session.eval("3 + 4")?;
```

`execute` returns a raw `Oop`:

```
let object_class = session.execute("Object")?;
```

`perform` returns a marshalled `Value`:

```
let seven = session.smallint_oop(7);
let printed = session.perform(seven, "printString", &[])?;
```

`perform_oop` returns a raw `Oop`:

```
let printed_oop = session.perform_oop(seven, "printString", &[])?;
let printed = session.fetch_string(printed_oop)?;
```

OOP and Value Conversion

`Value` covers nil, booleans, small integers, characters, fetched strings, and raw OOPs:

```
use gemstone_rs::Value;

let oop = session.value_to_oop(&Value::SmallInt(7))?;
let value = session.eval("true")?;
println!("{value:?}");
```

Helpers:

```
let nil = session.nil_oop();
let yes = session.bool_oop(true);
let seven = session.smallint_oop(7);
let text = session.new_string("hello")?;
let symbol = session.new_symbol("ExampleSymbol")?;
```

Globals

`global_get` and `global_put` use `UserGlobals`:

```
let text = session.new_string("hello from Rust")?;
session.global_put("GemStoneRsText", text)?;
let stored = session.global_get("GemStoneRsText")?;
println!("{}", session.fetch_string(stored)?);
```

Transactions

Use `transaction` when you want commit-on-success and abort-on-error:

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

Manual transaction primitives are also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
```

```
session.commit()?;
session.abort()?;
```

Object Mapping

Use `BridgeRoot` when Rust-owned payloads should live under a stable GemStone dictionary, by default `UserGlobals at: #GemStoneRsBridgeRoot`:

```
use gemstone_rs::{BridgeMapped, Config, Session};

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    status: String,
    amount: i64,
}

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let draft = BookingDraft {
    status: "draft".to_string(),
    amount: 100,
};

bridge_root.put_mapped("BookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("BookingDraft")?;
assert_eq!(loaded, draft);
bridge_root.commit()?;
```

Use `BridgeValue` when you need a dynamic inspection tree before choosing a typed Rust shape. Use `BTreeMap<String, T>` for string-keyed dictionary fields, and `BridgeValue::keyed_dictionary` when entries inside the GemStone dictionary must be Smalltalk symbols.

Use `Remote<T>` or `ObjectRef<T>` when an existing OOP should have a cached Rust view:

```
use gemstone_rs::{MaterializationProfile, Remote};

let oop = bridge_root.get_oop("BookingDraft")?;
let mut remote = Remote::<BookingDraft>::with_type(oop, "UserGlobals:BookingDraft")
    .with_profile(MaterializationProfile::deep(4));

let mut loaded = remote.refresh(&mut session)?.clone();
loaded.status = "confirmed".to_string();
remote.set_value(loaded);
remote.save(&mut session)?;
```

The mapping layer is explicit. Normal Rust field access does not call GemStone, and dropping a Rust value does not write it back. Reads and writes stay visible as `refresh`, `save`, `BridgeRoot::commit`, or `BridgeRoot::transaction`.

Export-Set Lifetime

Use `retain_oop` when a raw OOP must be held across calls and protected from GemStone-side collection:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Browser API

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Error Handling

Most APIs return `gemstone_rs::Result<T>`. Errors distinguish missing environment, missing config, GCI loading/calls, GemStone errors, illegal OOPs, unexpected typed codegen returns, and conversion issues.

```
match Config::from_env() {
    Ok(config) => println!("stone {}", config.stone),
    Err(err) => eprintln!("configuration error: {err}"),
}
```

The CLI exposes the same checks through `doctor`:

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs doctor --json
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
```

`env sample` prints a safe shell export template with password placeholders, and `env write` saves it to `.env.gemstone-rs` without overwriting existing files unless `--force` is used. `--env-file` is a global CLI option, so `doctor`, `eval`, `browse`, `inspect`, `bridge`, and `codegen` commands can load that file for one command. The non-live doctor form validates environment and GCI library loading. The live form also logs in and checks that `3 + 4` returns `7`. Human and JSON reports show which source selected `libgcirpc`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. `doctor --strict` fails when the stone or GCI source is only coming from defaults, which is better for CI. The JSON form is intended for automation and editor integrations, and includes the same remediation hints as the human report.

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

After installing the CLI, discover the same curated map without opening the repository docs:

```
gemstone-rs hello
gemstone-rs examples list
gemstone-rs examples show quickstart
gemstone-rs examples list --json
gemstone-rs examples map
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --batches
gemstone-rs compare all --status
gemstone-rs compare all --scorecard
gemstone-rs compare all --parity
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
gemstone-rs examples run codegen_preview --dry-run
gemstone-rs examples run axum_service --dry-run -- --routes
gemstone-rs examples run actix_service --dry-run -- --routes
gemstone-rs examples run python_native_adapter --dry-run
gemstone-rs examples run py_native_capabilities --dry-run
gemstone-rs examples run py_native_contract_fixture --dry-run
gemstone-rs examples run py_native_samples_fixture --dry-run
gemstone-rs examples run py_native_smoke_fixture --dry-run
gemstone-rs examples run py_native_migration_plan --dry-run
gemstone-rs examples run py_native_compatibility_fixture --dry-run
gemstone-rs examples run py_native_conformance_fixture --dry-run
gemstone-rs examples run py_native_handoff_bundle --dry-run
gemstone-rs examples run py_native_publish_receipt --dry-run
gemstone-rs examples run py_native_shared_core_gate --dry-run
gemstone-rs py-native capabilities --json
gemstone-rs py-native samples --json
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility --json
gemstone-rs py-native conformance --json
gemstone-rs py-native handoff --json
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
```

```
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
```

From a source checkout, `gemstone-rs examples run <name>` can launch the selected Cargo example or CLI-backed example directly. That includes the py-native fixture checks, so `examples list`, VS Code, and CI all point at the same adapter contract workflows. Use `--dry-run` when you only want to verify the command that would run. From an installed CLI, `gemstone-rs examples scaffold <name> [path]` writes a standalone Cargo project from embedded templates. Current scaffold templates include `quickstart`, `browser`, `bridge_root_mapping`, `derive_mapping`, `codegen_preview`, `codegen_workflow`, `codegen_discover`, `codegen_discover_mapping`, `profile_codegen_workflow`, `generated_wrapper_app`, `generated_mapping_app`, `http_service`, `session_worker_pool`, `py_native_pyo3_adapter`, `axum_service`, and `actix_service`; checked-in examples also include `bridge_value_inspection` for dynamic nested BridgeRoot read-back and `python_native_adapter` for the future `gemstone-py-native` wrapper contract. The `py_native_pyo3_adapter` scaffold writes a `pyproject.toml`, PyO3 `src/lib.rs`, and Python smoke tests that wrap the dependency-free `gemstone_rs::py_native` contract, including `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, and `handoff_json`, plus direct `NativeSession` methods for eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions. `eval_json` and `perform_json` return the stable `PyNativeValue` JSON shape for Python package code to decode. It also writes `python/gemstone_py_native_compat.py`, which wraps raw native OOP returns in `OopHandle` through `NativeCompatibilitySession` so package code can preserve existing Python return behavior while typed helpers and value conversion remain opt-in. It uses PyO3 0.28 so the generated starter remains compatible with current Python 3.14 interpreters, and `maturin` enables the `extension-module` Cargo feature only for Python extension builds. Source checkout verification also compiles and runs the scaffold against the local Rust core:

```
python3 scripts/check_py_native_pyo3_scaffold.py
```

The py-native examples also include a checked-in value/error samples fixture:

```
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native migration --json
```

That fixture gives wrapper CI concrete payloads for `nil`, booleans, small integers, characters, strings, symbols, OOPs, and structured errors. The `py-native migration --json` report tracks the `gemstone-py-native` shared-core path: wrapping `PyNativeSession`, preserving existing Python return behavior, keeping the live backend smoke green, and verifying published wheels from TestPyPI/PyPI installs. `py-native compatibility --json` and the checked-in `examples/py-native/gemstone-rs.py-native-compat.json` fixture document the generated compatibility shim: `NativeCompatibilitySession`, `OopHandle`, and the method-by-method mapping from Python-facing calls to raw native adapter calls. `py-native conformance --json` and

`examples/py-native/gemstone-rs.py-native-conformance.json` add the end-to-end wrapper target: extension module functions, raw `NativeSession` methods, compatibility shim methods, fixture paths, and scaffold files. `py-native handoff --json` and `examples/py-native/gemstone-rs.py-native-handoff.json` add the final downstream handoff bundle: every fixture path, schema, regeneration command, validation command, and acceptance check that `gemstone-py-native` needs before using the Rust core as its native backend. `py-native publish-receipt --json` and `examples/py-native/gemstone-rs.py-native-publish-receipt.json` record the verified TestPyPI and PyPI workflow runs, install commands, and package checks for the Rust-backed `gemstone-py-native` wheel release.
`py-native check-all` is the downstream shared-core gate. It validates the capabilities, samples, smoke, compatibility, conformance, handoff, and publish-receipt fixtures together, and `--json` gives CI or VS Code a single status report.

Aliases include `bridge`, `mapping`, `derive`, `codegen`, `discover`, `profiles`, `wrapper`, `framework`, `axum`, `actix`, and `http`. Scaffolds can include supporting project files; `profile_codegen_workflow` writes both `gemstone-rs.codegen` and `gemstone-rs.codegen-profiles.json`. `gemstone-rs examples map` is the Rust equivalent of `gemstone-examples plan3-map`: it groups crates, examples, docs, and gemstone-py reference points by feature stream. The same content is maintained in feature-map.md.

`gemstone-rs hello` is the no-live equivalent of `gemstone-examples hello`. Use it before configuring GemStone credentials when you only want to prove the CLI binary is installed and runnable.

Example Map

Feature	Command or path	What it demonstrates
Hello CLI	`gemstone-rs hello`	Verifies the installed CLI without GemStone credentials.
Scaffold quickstart	`gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart`	Creates a standalone Cargo quickstart project from the installed CLI.
Scaffold browser	`gemstone-rs examples scaffold browser ./gemstone-rs-browser`	Creates a standalone class-browser project from the installed CLI.
Scaffold BridgeRoot mapping	`gemstone-rs examples scaffold bridge_root_mapping ./gemstone-rs-bridge-root-mapping`	Creates a standalone BridgeRoot mapping project from the installed CLI.

Feature	Command or path	What it demonstrates
Scaffold derive mapping	`gemstone-rs examples scaffold derive_mapping ./gemstone-rs-derive-mapping`	Creates a standalone derive-mapping project from the installed CLI.
Scaffold codegen preview	`gemstone-rs examples scaffold codegen_preview ./gemstone-rs-codegen-preview`	Creates a standalone no-live codegen preview project from the installed CLI.
Scaffold codegen workflow	`gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow`	Creates a standalone no-live codegen preview/diff/check/generate project from the installed CLI.
Scaffold codegen discovery	`gemstone-rs examples scaffold codegen_discover ./gemstone-rs-codegen-discover`	Creates a standalone live discovery project from the installed CLI.
Scaffold mapping discovery	`gemstone-rs examples scaffold codegen_discover_mapping ./gemstone-rs-codegen-discover-mapping`	Creates a standalone live mapping discovery project from the installed CLI.
Scaffold profile codegen	`gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen`	Creates a standalone profile-driven codegen project with config and profile files.
Scaffold generated wrapper	`gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper`	Creates a standalone generated-style wrapper app from the installed CLI.
Scaffold generated mapping	`gemstone-rs examples scaffold generated_mapping_app ./gemstone-rs-generated-mapping`	Creates a standalone generated-style BridgeMapped app from the installed CLI.
Scaffold HTTP service	`gemstone-rs examples scaffold http_service ./gemstone-rs-http-service`	Creates a standalone Rust HTTP health-service project from the installed CLI.
Scaffold worker pool	`gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool`	Creates a standalone bounded SessionWorkerPool project from the installed CLI.
Scaffold PyO3 adapter	`gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter`	Creates a starter `gemstone-py-native` PyO3 crate over `gemstone_rs::py_native`.

Feature	Command or path	What it demonstrates
Scaffold Axum service	`gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service`	Creates a standalone Axum health-service project from the installed CLI.
Scaffold Actix service	`gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service`	Creates a standalone Actix Web health-service project from the installed CLI.
First login	`cargo run -p gemstone-rs --example hello_gemstone`	Reads env config, logs in, prints a session id, and evaluates `3 + 4`.
Quickstart	`cargo run -p gemstone-rs --example quickstart`	Eval, `global_put`, `global_get`, string fetch, cleanup.
Eval only	`cargo run -p gemstone-rs --example eval`	Minimal `Session::eval` shape.
Browser API	`cargo run -p gemstone-rs --example browser`	Dictionaries, protocols, methods, and source.
Live smoke cookbook	`cargo run -p gemstone-rs --example live_smoke_cookbook`	Login, eval, global round-trip, perform, and transaction checks in one run.
Transactions	`cargo run -p gemstone-rs --example transactions`	Commit-on-success and abort-on-error transaction wrapper.
Session worker	`cargo run -p gemstone-rs --example session_worker`	Dedicated-thread `SessionWorker` for web services and async runtimes.
Session worker pool	`cargo run -p gemstone-rs --example session_worker_pool`	Bounded round-robin pool of dedicated GemStone session workers.
Async worker facade	`cargo run -p gemstone-rs --example async_worker`	Awaitable `SessionWorkerPool` calls for async runtimes without moving `Session` across threads.
Python native adapter	`gemstone-rs py-native smoke --dry-run`; `cargo run -p gemstone-rs --example python_native_adapter -- --dry-run`	Dependency-free `py_native` contract used by the `gemstone-py-native` PyO3 bridge and smoke tests.

Feature	Command or path	What it demonstrates
Python native contract fixtures	`gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json`; `gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json`	Checked-in capability and smoke samples for wrapper CI and editor tooling.
Python native value/error samples	`gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-samples.json`	Concrete value and error payload samples for Python wrapper translation tests.
Python native migration plan	`gemstone-rs py-native migration --json`; `gemstone-rs examples run py_native_migration_plan --dry-run`	Machine-readable checklist for completing the gemstone-py-native shared-core migration.
Python native compatibility fixture	`gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-compat.json`; `gemstone-rs examples run py_native_compatibility_fixture --dry-run`	Checks the Python package-layer return policy and shim method mapping.
Python native conformance fixture	`gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-conformance.json`; `gemstone-rs examples run py_native_conformance_fixture --dry-run`	Checks the PyO3 module/session/shim surface that a real `gemstone-py-native` wrapper should expose.
Python native handoff bundle	`gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-handoff.json`; `gemstone-rs examples run py_native_handoff_bundle --dry-run`	Checks the downstream `gemstone-py-native` handoff manifest across contract, smoke, compatibility, conformance, and acceptance criteria.
Python native publish receipt	`gemstone-rs py-native check-publish-receipt examples/py-native/gemstone-rs.py-native-publish-receipt.json`; `gemstone-rs examples run py_native_publish_receipt --dry-run`	Checks the verified TestPyPI/PyPI workflow runs and install receipts for the Rust-backed native wheel release.
Python native shared-core gate	`gemstone-rs py-native check-all`; `gemstone-rs examples run py_native_shared_core_gate --dry-run`	Checks every checked-in py-native fixture in one downstream CI gate.

Feature	Command or path	What it demonstrates
Python native examples runner	`gemstone-rs examples run py_native_capabilities --dry-run`; `gemstone-rs examples run py_native_contract_fixture --dry-run`; `gemstone-rs examples run py_native_smoke_fixture --dry-run`; `gemstone-rs examples run py_native_migration_plan --dry-run`; `gemstone-rs examples run py_native_compatibility_fixture --dry-run`; `gemstone-rs examples run py_native_conformance_fixture --dry-run`; `gemstone-rs examples run py_native_handoff_bundle --dry-run`; `gemstone-rs examples run py_native_publish_receipt --dry-run`; `gemstone-rs examples run py_native_shared_core_gate --dry-run`	Exposes py-native fixture workflows through the same examples catalog as Cargo examples.
OOP/value conversion	`cargo run -p gemstone-rs --example oop_values`	`Value`, `Oop`, strings, symbols, and export-set retention.
BridgeRoot mapping	`cargo run -p gemstone-rs --example bridge_root_mapping`	MagLev-style bridge-root storage with explicit `BridgeValue` mapping.
Derive mapping	`cargo run -p gemstone-rs --example derive_mapping`	`#[derive(BridgeMapped)]`, symbol keys, nested structs, vectors, maps, and BridgeRoot transactions.
BridgeValue inspection	`cargo run -p gemstone-rs --example bridge_value_inspection`	Reads nested BridgeRoot dictionaries and arrays back as dynamic `BridgeValue` trees with shape reports.
Remote object mapping	`cargo run -p gemstone-rs --example remote_object_mapping`	Uses `Remote<T>` to refresh, edit, and explicitly save a mapped dictionary OOP.
Bridge mapping preview	`cargo run -p gemstone-rs --example bridge_mapping_preview`	Infers a starter `BridgeMapped` codegen config from a nested `BridgeValue` tree.
Offline codegen	`cargo run -p gemstone-rs --example codegen_preview`	

Feature	Command or path	What it demonstrates
		Generates wrappers from the sample config without a live stone.
Codegen workflow	`cargo run -p gemstone-rs --example codegen_workflow`	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	`cargo run -p gemstone-rs --example codegen_discover`	Connects to a live stone and discovers a starter config for `Object`.
Mapping discovery	`cargo run -p gemstone-rs --example codegen_discover_mapping`	Connects to a live stone and proposes a `BridgeMapped` config.
Generated wrapper app	`cargo run -p gemstone-rs --example generated_wrapper_app`	Uses checked-in generated wrappers to call `Object>>printString`.
Generated mapping app	`cargo run -p gemstone-rs --example generated_mapping_app`	Uses codegen-created `BridgeMapped` structs with `BridgeRoot`.
Generated wrapper compile check	`cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml`	Imports the checked-in generated wrappers as a separate crate and runs generated metadata tests.
Codegen files	`examples/codegen/`	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	`examples/tooling/explorer.md`	Local explorer startup and endpoint checks.
VS Code tooling	`examples/tooling/vscode-workbench.md`	Sidebar browsing, codegen actions, and explorer launch.
CLI browser walkthrough	`examples/tooling/cli-browser-walkthrough.md`	Terminal-only browse workflow.
HTTP service	`cargo run -p gemstone-rs --example http_service -- --routes`	Standard-library web service with `/`, `/health/local`, and `/health/gemstone`.
Axum service	`cargo run --manifest-path examples/axum-service/Cargo.toml -- --routes`	

Feature	Command or path	What it demonstrates
		Checked Axum route shape with `gemstone-rs-axum`, `SessionWorkerPool`, and shared `gemstone_rs::web` health responses.
Actix service	`cargo run --manifest-path examples/actix-service/Cargo.toml -- --routes`	Checked Actix route shape with `gemstone-rs-actix`, `SessionWorkerPool`, and shared `gemstone_rs::web` health responses.

The Axum and Actix services use the graceful health-pool startup path. They can start with missing credentials, serve `/` and `/health/local`, and return a `503` JSON error from `/health/gemstone` until the stone is configured. Verify the route contract with:

```
python3 scripts/framework_route_smoke.py
scripts/live_smoke.sh --dry-run
scripts/live_smoke.sh
```

The same smoke check asserts diagnostic and request-trace headers from the adapters: `x-gemstone-rs-adapter`, `x-gemstone-rs-route`, `x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path`. It also asserts the lifecycle headers `x-gemstone-rs-request-lifecycle: received,handled` and `x-gemstone-rs-request-duration-us`, which give tests and local proxies a framework-neutral way to confirm the packaged handler ran. The checked services also add `x-gemstone-rs-example-middleware: axum` or `x-gemstone-rs-example-middleware: actix`, `x-gemstone-rs-service`, `x-gemstone-rs-service-version`, `cache-control: no-store`, and `x-content-type-options: nosniff`. The smoke script asserts those headers so application middleware and a small production-style cache/security policy stay covered. Use `scripts/live_smoke.sh` when GemStone credentials are available and `/health/gemstone` should be required to return `{"result":7}` as part of the same live lane that runs Rust tests and live examples.

Suggested Learning Order

1. `gemstone-rs hello`
2. `hello_gemstone`
3. `quickstart`
4. `browser`
5. `live_smoke_cookbook`
6. `transactions`
7. `session_worker`
8. `session_worker_pool`

9. `async_worker`
10. `oop_values`
11. `bridge_root_mapping`
12. `derive_mapping`
13. `bridge_value_inspection`
14. `codegen_preview`
15. `codegen_workflow`
16. `generated_wrapper_app`
17. `generated_mapping_app`
18. `codegen_discover`
19. `codegen_discover_mapping`
20. `examples/codegen/`
21. `examples/tooling/cli-browser-walkthrough.md`
22. `examples/tooling/explorer.md`
23. `examples/tooling/vscode-workbench.md`
24. `http_service`
25. `examples/axum-service/`
26. `examples/actix-service/`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example live_smoke_cookbook
login ok: session <number>
eval ok: SmallInt(7)
global round-trip ok
perform ok: 7
transaction commit/abort ok

$ cargo run -p gemstone-rs --example bridge_root_mapping
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
```

```
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }

$ cargo run -p gemstone-rs --example bridge_value_inspection
dynamic BridgeValue: Dictionary({"customer": Dictionary(...), "items": Array(...),
"note": Nil, "state": Symbol("ready")})
bridge root identity: <number>
bridge root key count: <number>

$ cargo run -p gemstone-rs --example remote_object_mapping
remote loaded: BookingDraft { status: "draft", amount: 100, labels: {"source":
"remote-example"} }
remote saved: BookingDraft { status: "confirmed", amount: 100, labels: {"source":
"remote-example"} }

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }

$ cargo run -p gemstone-rs --example http_service -- --routes
gemstone-rs HTTP service example
  GET /
  GET /health/local
  GET /health/gemstone
```

The mapping examples use `BTreeMap<String, T>` for string-keyed dictionary metadata. Use `BridgeValue::keyed_dictionary` when the entries inside the GemStone dictionary must be symbol-keyed for Smalltalk code.

Mapping examples cover progressively stronger layers:

Layer	Example	Purpose
`BridgeValue`	`bridge_value_inspection`	Inspect a live nested dictionary/array shape before committing to a Rust type.
`BridgeMapped`	`bridge_root_mapping`	Store and read typed dictionary-backed payloads under `GemStoneRsBridgeRoot`.
`#[derive(BridgeMapped)]`	`derive_mapping`	

Layer	Example	Purpose
		Remove manual mapping boilerplate while keeping field keys and key types explicit.
`Remote<T>`	`remote_object_mapping`	Refresh, edit, and explicitly save an OOP-backed mapped value.
codegen mapping	`generated_mapping_app`	Use generated mapping structs and wrapper tests from checked-in config.
connector metadata	`examples/codegen/connector-mapping.codegen`	Record Smalltalk selectors and return shapes beside mapped Rust fields for tool review.

The examples intentionally do not use transparent persistence. Normal Rust field access is local-only; GemStone reads and writes happen at visible calls such as `refresh(&mut session)`, `bridge_root.transaction(...)`, and `remote.save(&mut session)`.

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

The checked-in codegen sample also demonstrates typed arguments. A config line such as `method = Object>>perform: | args=selector:Symbol` generates a Rust method that accepts `selector: impl AsRef<str>` and creates the GemStone symbol before dispatch. For application wrappers, use `SmallInt`, `String`, `Symbol`, `Bool`, or the default `Oop` depending on how much conversion you want at the generated boundary. Typed returns cover the same common value shapes: `return=Symbol` fetches the GemStone Symbol as a Rust `String`, while `return=SmallInt`, `return=Bool`, `return=String`, and `return=Oop` narrow the result at the wrapper boundary.

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```
cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --json
```

```
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
gemstone-rs bridge sample-config BookingDraft
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain-profile --json default examples/codegen/gemstone-
rs.codegen-profiles.json
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --batches
gemstone-rs compare all --status
gemstone-rs compare all --scorecard
gemstone-rs compare all --parity
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
gemstone-rs examples run axum_service --dry-run -- --routes
gemstone-rs examples run actix_service --dry-run -- --routes
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Codegen Workflow

```
cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
```

```
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object
```

Use the checked-in explorer profile sample when you want a repeatable browser workflow for codegen:

```
cat examples/codegen/gemstone-rs.codegen-profiles.json
gemstone-rs-explorer --port 8787 --codegen-root .
```

In the explorer, click `Load Project Profiles` with `examples/codegen/gemstone-rs.codegen-profiles.json` as the project profile file, then select `default`, `object-wrapper`, `bridge-mapping`, or `connector-mapping`.

Validate profile files before committing them:

```
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

Generate a starter config from a live stone:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated wrapper example after checking the generated file into the repository:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

VS Code

Install the workbench from:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, profile-driven codegen actions, Generate Mapping Config, Preview BridgeRoot, List BridgeRoot Keys, Put BridgeRoot String, Put BridgeRoot Symbol, Put BridgeRoot SmallInt, Put BridgeRoot Bool, Remove BridgeRoot Key, Run

Generated Mapping Example, Load Project Profiles, Save Project Profiles, Export Codegen Profile, Show Sample Project Profiles, Create Project Profiles, Validate Project Profiles, List Project Profiles, Show Project Profile, Resolve Project Profile, Check Project Profiles, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a local explorer workflow with screenshots
- broader framework coverage and richer real-application middleware examples
- a richer class browser walkthrough with captured output

gemstone-rs Feature Map

This map is the Rust equivalent of `gemstone-examples plan3-map` in `gemstone-py`: it ties each feature stream to the crates, examples, docs, and Python reference point that inspired it.

Run it from the CLI:

```
gemstone-rs hello
gemstone-rs compare gemstone-py
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --batches
gemstone-rs compare all --status
gemstone-rs compare all --scorecard
gemstone-rs compare all --parity
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
gemstone-rs examples map
gemstone-rs examples map --json
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
```

`hello` is the no-live sanity check that mirrors `gemstone-examples hello`. `compare gemstone-py` prints a compact version of the Python/Rust comparison guide. `compare gemstone-js` remains available as archived background reference, but active planning now focuses on gemstone-rs. The `--status` forms give the shortest answer with the direct recommendation, parity score, remaining batch count, next batch when one exists, top gap, and follow-up commands. The `--scorecard` forms print the shortest decision view: when to use each project, current strengths, the remaining batch count, the next batch when one exists, and the top gap. The `--parity` forms print area-by-area maturity scores for API, web, async, codegen, explorer, release, and native-core work. The `--gaps` forms print remaining parity gaps with next actions and verification commands. The `--next` forms print the first recommended batch and top priority gap. The `--totals` forms print only the batch/hour totals for planning and CI. The `--batches` forms answer how much work remains, including batch counts, hour ranges, outcomes, and verification commands. Use `compare all` with the same flags to print the active gemstone-rs track; `compare all --batches` reports **0 batches** and roughly **0-0 hours** after the Rust-backed `gemstone-py-native` TestPyPI/PyPI wheel verification passed. `examples scaffold` creates standalone Cargo projects from installed templates for quickstart, browser, BridgeRoot mapping, derive mapping, generated wrappers, live discovery, profile-driven codegen, standard HTTP, worker-pool, Axum, and Actix workflows. JSON output is intended for CI, docs checks, and editor tooling. Compare reports are covered by `schemas/gemstone-rs.compare.schema.json`, so release scripts and editor panels can validate the same summary, status, scorecard, parity, gap, next-action, totals, and batch-plan shapes that the CLI prints.

Streams

Stream	Feature	Rust surface	Examples	Docs	gems
1	Runtime and GCI loading	`gemstone-gci`, `gemstone-rs::Config`	`hello`, `hello_gemstone`, `quickstart`	`docs/setup-guide.md`, `docs/performance-safety.md`	`gems` native
2	Safe sessions and transactions	`gemstone-rs::Session`, `SessionWorker`, `SessionWorkerPool`, `TransactionGuard`	`quickstart`, `transactions`, `session_worker`, `session_worker_pool`, `live_smoke_cookbook`	`docs/user-manual.md`, `docs/cookbook.md`	`GemS` `Sess` transa
3	Browser and inspection	`gemstone-rs::browser`, CLI `browse`, `gemstone-rs-explorer`	`browser`, `tooling/cli-browser-walkthrough.md`	`docs/user-manual.md`, `docs/explorer.md`	`gems` `pyth` `datab`

Stream	Feature	Rust surface	Examples	Docs	gems
4	OOP and value handling	`Oop`, `Value`, export-set helpers	`oop_values`	`docs/user-manual.md`, `docs/performance-safety.md`	Mana typed
5	BridgeRoot object mapping	`gemstone-rs::bridge`, `gemstone-rs-macros`	`bridge_root_mapping`, `derive_mapping`, `bridge_value_inspection`, `remote_object_mapping`, `generated_mapping_app`	`docs/object-mapping.md`, `docs/cookbook.md`	`Smal` `Pers` exam
6	Typed codegen	`gemstone-rs::codegen`, CLI `codegen` and `profile`	`codegen_preview`, `codegen_workflow`, `codegen_discover`, `profile_codegen_workflow`, `generated_wrapper_app`	`docs/codegen.md`, `docs/profile-schema.md`	`gems` `type` `codeg`
7	Explorer workflow	`gemstone-rs-explorer`	`tooling/explorer.md`	`docs/explorer.md`, `docs/screenshots.md`	`pyth` `datab`

Stream	Feature	Rust surface	Examples	Docs	gems
8	VS Code workbench	`vscode-gemstone-rs-workbench`	`tooling/vscode-workbench.md`	`docs/vscode-workbench.md`	`gems`
9	Rust web services	`gemstone-rs::web`, `gemstone-rs-axum`, `gemstone-rs-actix`, `SessionWorkerPool`, async worker futures, checked Axum/Actix services, and installed scaffolds	`session_worker`, `session_worker_pool`, `async_worker`, `http_service`, `examples/axum-service`, `examples/actix-service`, `framework_route_smoke.py`, `examples scaffold session_worker_pool/ axum_service/ actix_service`	`docs/examples-guide.md`, `docs/cookbook.md`	FastA exam

Stream	Feature	Rust surface	Examples	Docs	gems
10	Release and verification	`scripts`, `Makefile`, GitHub Actions	Release verification commands	`docs/release-checklist.md`	PyPI/ VSIX
11	Shared native core	`gemstone-gci`, `gemstone-rs::py_native`, `gemstone-py-native` wrapper	`python_native_adapter`, `py_native_pyo3_adapter` scaffold, downstream `gemstone-py-native` bridge	`docs/shared-core-integration.md`	`gems`

What This Says Compared With gemstone-py

`gemstone-rs` is strongest where Rust is naturally valuable: direct native GemStone access, explicit OOP/value handling, typed generated wrappers, BridgeRoot mapping, and CLI/explorer tooling that can run without Python in the process.

`gemstone-py` remains stronger where Python already has mature product shape: broader web framework adapters, async examples, the database explorer, package extras, and broader release surfaces.

The projects should converge by sharing the Rust native core underneath `gemstone-py-native`, while keeping the Python and Rust APIs idiomatic for their respective users.

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env()?)?;
println!("{:?}", session.eval("3 factorial")?);
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify `3 + 4`

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read `UserGlobals`

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env()?)?;
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust")?;
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;

let result: gemstone_rs::Result<()> = session.transaction(|session| {
    let value = session.new_string("will abort")?;
    session.global_put("GemStoneRsAborted", value)?;
    Err(Error::IllegalOop {
        operation: "intentional abort",
    })
});

assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
let yes = session.bool_oop(true);
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello")?;
let text = session.fetch_string(string_oop)?;
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("retained OOP {}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Mapping Quick Decision Table

Use the lowest layer that matches the job:

Need	Use
Direct selector sends or raw identity	`Oop` plus `Session::perform`
Explore a live value before typing it	`BridgeValue` and shape reports
Store stable Rust payloads in GemStone	`BridgeMapped` under `BridgeRoot`
Avoid hand-written mapping impls	`#[derive(BridgeMapped)]`
Keep a cached view of an existing OOP	`Remote<T>` / `ObjectRef<T>`
Make selector/mapping APIs repeatable	codegen config and checked-in generated wrappers

The mapping layer is explicit. Reads and writes should be visible in the code:

```
let loaded = remote.refresh(&mut session)?.clone();
remote.set_value(loaded);
remote.save(&mut session)?;
```

Do not rely on implicit write-back or hidden field-level network calls. Use `BridgeRoot::transaction` or `Session` transaction helpers when a group of changes must commit or abort together.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};
use std::collections::BTreeMap;

let mut session = Session::login(Config::from_env()?)?;
let labels = BTreeMap::from([("source".to_string(), "cookbook".to_string())]);
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
    ("labels".to_string(), BridgeValue::from(labels)),
]);

let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

The default root is `UserGlobals at: #GemStoneRsBridgeRoot`.

Recipe 10B: Round-Trip a Typed BridgeRoot Map Field

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
```

```
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            labels: dictionary.at_map("labels")?,
        })
    }
}

let mut session = Session::login(Config::from_env()?)?;
let draft = BookingDraft {
    name: "Tariq".to_string(),
    labels: BTreeMap::from([("source".to_string(), "cookbook".to_string())]),
};

let mut bridge_root = session.bridge_root()?;
bridge_root.put_mapped("CookbookBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("CookbookBookingDraft")?;
assert_eq!(loaded.labels["source"], "cookbook");

bridge_root.put_string("CookbookStatus", "ready")?;
bridge_root.put_vec("CookbookTags", &["priority".to_string(), "demo".to_string()])?;
bridge_root.put_map("CookbookLabels", &draft.labels)?;
assert_eq!(bridge_root.get_string("CookbookStatus")?, "ready");
let tags: Vec<String> = bridge_root.get_vec("CookbookTags")?;
assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
let labels: BTreeMap<String, String> = bridge_root.get_map("CookbookLabels")?;
assert_eq!(labels["source"], "cookbook");

bridge_root.put_map_with_key_type("CookbookLabelsSymbol", BridgeKeyType::Symbol,
&draft.labels)?;
let symbol_labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("CookbookLabelsSymbol",
BridgeKeyType::Symbol)?;
assert_eq!(symbol_labels["source"], "cookbook");
```

Use map fields for string-keyed metadata or lookup tables. Use nested `BridgeMapped` structs when the related value has a stable domain shape.

Recipe 10C: Store a Symbol-Keyed Dictionary

Use `BTreeMap<String, T>` for string-keyed metadata. When the GemStone dictionary entries themselves need symbol keys, build a keyed dictionary explicitly:

```
use gemstone_rs::{BridgeKey, BridgeKeyType, BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::keyed_dictionary([
    (BridgeKey::symbol("status"), BridgeValue::from("ready")),
    (BridgeKey::symbol("amount"), BridgeValue::from(100_i64)),
    (BridgeKey::string("externalId"), BridgeValue::from("web-42")),
]);

bridge_root.put("CookbookSymbolDictionary", payload)?;

let mut dictionary = bridge_root.get_dictionary("CookbookSymbolDictionary")?;
assert_eq!(
    dictionary.at_string_with_key_type("status", BridgeKeyType::Symbol)?,
    "ready"
);
assert_eq!(
    dictionary.at_smallint_with_key_type("amount", BridgeKeyType::Symbol)?,
    100
);
assert_eq!(dictionary.at_string("externalId")?, "web-42");
```

`put_map_with_key_type` changes the key used to store the map in the containing dictionary. It does not change the entry keys inside the stored map. Use `BridgeValue::keyed_dictionary` for symbol-keyed entries inside the value.

Recipe 10D: Refresh and Save a Remote Mapped Object

`Remote<T>` is an explicit object reference. It stores an `Oop` and type metadata, but it only talks to GemStone when you pass `&mut Session`.

```
use gemstone_rs::{BridgeMapped, Config, MaterializationProfile, Remote, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    status: String,
    amount: i64,
    labels: BTreeMap<String, String>,
}

let mut session = Session::login(Config::from_env()?)?;
let key = "CookbookRemoteBooking";

let initial = BookingDraft {
    status: "draft".to_string(),
```

```
    amount: 100,
    labels: BTreeMap::from([("source".to_string(), "cookbook".to_string())]),
};

let oop = {
    let mut bridge_root = session.bridge_root()?;
    bridge_root.put_mapped(key, &initial)?;
    bridge_root.get_oop(key)?
};

let mut remote = Remote::<BookingDraft>::with_type(oop, "UserGlobals:BookingDraft")
    .with_profile(MaterializationProfile::deep(4));
let mut loaded = remote.refresh(&mut session)?.clone();
loaded.status = "confirmed".to_string();
remote.set_value(loaded);
remote.save(&mut session)?;
```

There is no hidden save-on-drop. `save(&mut session)` is the persistence point.

Recipe 11: Browse Dictionaries

```
use gemstone_rs::browser::Browser;

let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{dictionary}");
}
```

Recipe 12: Browse a Class

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Diagnose a New Machine

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
```

The first command checks environment and GCI loading, including whether `libgcirpc` came from explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. The second command logs in and verifies that `3 + 4` returns `7`. The JSON form is useful in CI or editor tooling.

Recipe 14: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 15: Inspect BridgeRoot From the CLI

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
```

Use this when object-mapping examples or generated mappings write payloads under `GemStoneRsBridgeRoot`. `bridge keys` is the quickest way to see what is currently stored before inspecting a specific payload. The `put-*` shortcuts and `bridge remove` are useful live-write smoke tests because they commit one explicit BridgeRoot change at a time. Use generic `bridge put --type ...` when you want the value type to be data-driven in a script.

Recipe 16: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 17: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 18: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 19: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 19A: Run a Minimal Rust HTTP Health Service

```
cargo run -p gemstone-rs --example http_service -- --routes
cargo run -p gemstone-rs --example http_service -- --port 3000
```

From a second terminal:

```
curl -i http://127.0.0.1:3000/
curl -i http://127.0.0.1:3000/health/local
curl -i http://127.0.0.1:3000/health/gemstone
```

The example uses only the Rust standard library. It proves the service shape and GemStone health-check flow without pulling web framework dependencies into the core crate. `gemstone-py` is still ahead for batteries-included FastAPI, Litestar, and Django adapters; `gemstone-rs` now has a direct Rust service smoke path plus packaged Axum and Actix adapter crates:

```
cargo run --manifest-path examples/axum-service/Cargo.toml -- --routes
cargo run --manifest-path examples/actix-service/Cargo.toml -- --routes
```

The Axum and Actix services now start a bounded `SessionWorkerPool`, then use `gemstone-rs-axum` or `gemstone-rs-actix` to expose `/`, `/health/local`, and `/health/gemstone` without copying handler code.

For local development, the framework examples use the non-failing health-pool startup path. The server can start before credentials are configured: `/` and `/health/local` stay available, while `/health/gemstone` returns a `503` JSON error until the stone is reachable. The route smoke script checks that behavior without needing a live stone:

```
python3 scripts/framework_route_smoke.py
scripts/live_smoke.sh --dry-run
scripts/live_smoke.sh
```

The adapters also return `x-gemstone-rs-adapter`, `x-gemstone-rs-route`, `x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path` headers, plus `x-gemstone-rs-request-lifecycle: received,handled` and `x-gemstone-rs-request-duration-us`. The smoke script asserts those headers so a proxy, load balancer, or test can distinguish the framework adapter, route, request, lifecycle, and handler duration that produced the response. It also asserts `x-gemstone-rs-example-middleware: axum` or `x-gemstone-rs-example-middleware: actix`, `x-gemstone-rs-service`, `x-gemstone-rs-service-version`, `cache-control: no-store`, and `x-content-type-options: nosniff` from the checked services. That proves the packaged routes still compose with normal framework middleware and gives the examples a small production-style cache/security header policy. In live mode the same script requires `/health/gemstone` to reach the stone and return `{"result":7}` for both adapters; the manual CI job runs that path with GemStone secrets.

Use `SessionWorker` when the application wants a reusable dedicated GemStone session lane instead of opening a new session inside each blocking route:

```
use gemstone_rs::{Config, SessionWorker, Value};

let worker = SessionWorker::start(Config::from_env()?)?;
assert_eq!(worker.eval("3 + 4")?, Value::SmallInt(7));
worker.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

Use `SessionWorkerPool` when the service should keep a fixed number of GemStone sessions warm and dispatch calls round-robin:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};
```

```
let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval("3 + 4")?, Value::SmallInt(7));
pool.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

The same pool exposes awaitable calls for async runtimes:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval_async("3 + 4").await?, Value::SmallInt(7));
pool.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

The installed CLI can also scaffold this shape:

```
gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool
```

The dependency-free `gemstone_rs::web` helpers now use the same async worker facade in Axum and Actix health handlers, so framework routes no longer need to wrap GemStone health checks in framework-specific blocking helpers.

Recipe 20: Check the Python Native Adapter Contract

Use this when you are preparing `gemstone-py-native` to wrap the Rust core:

```
gemstone-rs py-native capabilities
gemstone-rs py-native capabilities --json
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json
gemstone-rs py-native samples --json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
gemstone-rs py-native smoke --dry-run
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility --json
gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-
compat.json
gemstone-rs py-native conformance --json
gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-
conformance.json
gemstone-rs py-native handoff --json
gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-
handoff.json
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-publish-receipt examples/py-native/gemstone-rs.py-native-
```

```
publish-receipt.json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
```

The CLI command prints the contract version, threading rule, supported operations, value kinds, error kinds, and OOP constants. The fixture checks compare the checked-in capability, value/error sample, and dry-run smoke JSON against the shared core renderer. `py-native samples --json` gives Python wrapper CI concrete payloads for `nil`, booleans, small integers, characters, strings, symbols, OOPs, and structured errors. The smoke command checks capabilities, OOP constants, value conversion, config error mapping, and structured error mapping without a live stone when `--dry-run` is passed. The `py-native migration --json` report turns the shared-core path into a concrete checklist for `gemstone-py-native`: wrap `PyNativeSession`, preserve the Python API, keep live backend smoke green, and verify published wheels from TestPyPI/PyPI. `py-native compatibility --json` adds a method-by-method map for the generated Python shim, including `OopHandle` return points and typed helper methods. The `py-native conformance --json` report adds the integration target: generated module functions, raw `NativeSession` methods, compatibility shim methods, fixture files, and scaffold files. `py-native handoff --json` adds the final downstream manifest tying the contract, samples, smoke, migration, compatibility, conformance, and acceptance checks together. `py-native publish-receipt --json` records the verified TestPyPI and PyPI workflow runs, install commands, and package checks for the Rust-backed `gemstone-py-native` wheel release. The `py-native check-all` gate validates all checked-in py-native fixtures together, which is the shortest command to put in downstream `gemstone-py-native` CI before publishing native wheels. The PyO3 scaffold writes a minimal `gemstone_py_native` extension module with `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, `handoff_json`, and an unsendable `NativeSession` wrapper over eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions. The wrapper also exposes `eval_json` and `perform_json` so the Python package layer can decode the stable `PyNativeValue` JSON shape instead of inferring native values itself. It also writes `python/gemstone_py_native_compat.py`, a Python package-layer shim with `NativeCompatibilitySession`, `OopHandle`, and a `compatibility_report()` that documents the return policy: object identity returns handles, raw native OOPs stay below the package boundary, value-returning methods become dictionaries, and typed helpers are opt-in. The live example logs in, evaluates `3 + 4`, performs `printString`, and round trips a `UserGlobals` string through `PyNativeSession`:

```
gemstone-rs py-native smoke
cargo run -p gemstone-rs --example python_native_adapter
```

The important point is architectural: Python should wrap `gemstone_rs::py_native`; it should not duplicate dynamic GCI loading or raw session calls.

Recipe 21: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 22: Add a Local Explorer Auth Token

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --port 8787 --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

VS Code users can run `GemStone RS: Generate Explorer Auth Token`; the workbench stores the token in `gemstoneRs.explorerAuthToken`, launches the explorer with `--auth-token-env GEMSTONE_RS_EXPLORER_TOKEN`, and opens token-aware browser/webview URLs.

Recipe 23: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 24: Use the VS Code Workbench With a Checkout

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 25: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.2
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	`python -m pip install gemstone-py`	`cargo add gemstone-rs`
Native acceleration	`python -m pip install "gemstone-py[fast]"`	Built into the Rust GCI path once `libgcirpc` is available
Examples from source	`python -m pip install -e ".[examples]"`	`cargo run -p gemstone-rs --example quickstart`
Example discovery/run	`gemstone-examples hello`, `list`, `plan3-map`, and launch commands	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, source-checkout `run`, and installed `scaffold` templates
CLI tooling	Python modules and examples	`cargo install gemstone-rs-cli`
Local explorer	`python-gemstone-database-explorer`	`cargo install gemstone-rs-explorer`
VS Code	`gemstone-py Workbench`	`gemstone-rs Workbench`

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
Example-driven onboarding	gemstone-py today, gemstone-rs catching up	gemstone-py has broader runnable examples; gemstone-rs now has no-live hello, a direct comparison command, an installed CLI example index, feature map, source-checkout runner, and standalone project scaffolds.
VS Code database/ codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added

Capability	gemstone-py	gemstone-rs
Native bridge	PyO3 extension path	Rust GCI crate, safe API, and dependency-free `py_native` adapter contract
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Dedicated-thread `SessionWorker`, bounded `SessionWorkerPool`, and dependency-free awaitable worker futures
Web frameworks	FastAPI, Litestar, Django examples	Shared `gemstone_rs::web` health helpers, standard-library HTTP, `SessionWorkerPool`, packaged `gemstone-rs-axum` / `gemstone-rs-actix` adapters, checked Axum/Actix examples, request-trace, lifecycle-duration, and production-style service/cache/security middleware headers
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate, live discovery, typed argument conversion, typed return helpers, and profile scaffolds
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, example index, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported

Feature	gemstone-py status	gemstone-rs status
`3 + 4 == 7` smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported
`perform` and `perform_oop`	Supported	Supported
Commit/abort	Supported	Supported
Dedicated session worker	Supported through Python web integration patterns	Supported through `SessionWorker` and `SessionWorkerPool`
OOP handles/export set	Supported	Supported
Browser dictionaries/ classes/protocols/ methods/source	Supported	Supported through `Browser`, CLI, and explorer
Object mapping	Mature Python ergonomics around dictionaries and app objects	Explicit Rust layers: `Oop`, `BridgeValue`, `BridgeMapped`, `BridgeRoot`, `Remote<T>`, materialization profiles, and codegen mapping
Generated wrappers	Supported	Supported for selected classes/methods
Generated typed arguments	Python naturally passes Python values	Supported through `args=name:Type` for `SmallInt`, `String`, `Symbol`, `Bool`, and explicit `Oop`
Codegen diff/check/ generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API and installed scaffolds
FastAPI demo	Supported	Not applicable
Litestar demo	Supported	Not applicable
Rust HTTP service demo	Not applicable	Supported through `cargo run -p gemstone-rs -- example http_service -- --port 3000`

Feature	gemstone-py status	gemstone-rs status
Axum/Actix demo	Not applicable	Axum and Actix services supported through `gemstone-rs-axum`, `gemstone-rs-actix`, `examples/axum-service`, and `examples/actix-service`
Installed examples index	`gemstone-examples hello`, `list`, `plan3-map`	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, `run --dry-run`, `scaffold`, plus JSON for tooling
Database explorer	Mature Python app	Rust explorer has browser UI, local API, profile status, codegen diff/explain, editable generated output, nested BridgeValue rendering, comparison workflows, and committed visual assets; still less polished
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and an embedded explorer webview with structured setup, profile, codegen, diff, editable generated output, BridgeRoot nested values, comparison panels, and Marketplace/GitHub visuals

Actionable Gap Report

The CLI can print the remaining gemstone-py parity gaps directly:

```
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --gaps --json
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --status --json
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --scorecard --json
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --parity --json
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --next --json
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --totals --json
gemstone-rs compare gemstone-py --batches
gemstone-rs compare gemstone-py --batches --json
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
```

`compare all --batches` now reports only active gemstone-rs work. The gemstone-js comparison remains available as archived background reference, but it is no longer part of the active remaining-work estimate. Active remaining work is **0 batches**, roughly **0-0 hours** total after Rust-backed `gemstone-py-native` wheel verification passed.

The report is intentionally action-oriented. Each row names the gemstone-py strength, the gemstone-rs gap, the next implementation step, and the command or test that should verify it.

Use `--status` when you want the shortest answer. It combines the direct recommendation, parity score, remaining batch count, next batch when one exists, top gap, and follow-up commands in one report.

Use `--scorecard` when you want the decision answer instead of the full gap report. It summarizes where gemstone-py is stronger today, where gemstone-rs is stronger today, when to choose each project, how many batches remain, and the next recommended batch.

Use `--parity` when you want a measured maturity view. It scores each area out of five and shows the current leader, status, and next action. The current Rust/Python parity score is **gemstone-py 30/35** and **gemstone-rs 30/35**: Rust is at parity or ahead for core sessions, async worker boundaries, codegen/mapping, release verification, and the shared native-core direction; Python remains ahead for framework breadth and explorer polish.

Priority	Area	What gemstone-py has today	gemstone-rs next action
P2	Explorer product polish	The Python database explorer is the richer class browser and product reference.	Keep Marketplace/GitHub screenshots current and run explorer/API smoke tests after UI changes.
P1	Installed example experience	`gemstone-examples` launches installed examples without a source checkout.	Expand `gemstone-rs examples scaffold` to explorer-integrated projects and richer generated wrapper profile variants.
P2	Web framework adapters	FastAPI, Litestar, and Django examples are first-class.	Add more Rust framework examples only when a real service needs them.
P2	Async lifetime depth	gemstone-py has async examples and FastAPI integration.	Add deeper cancellation, shutdown, and lifetime tests around async worker futures.
P2	Release lane discipline	gemstone-py has mature PyPI/TestPyPI/native wheel/VSIX release lanes.	Keep running the Release and Post-Release Verify workflows after each publish.

Use `--next` when you only want the first recommended implementation step:

```
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --next --json
```

Remaining Work Batches

The batch plan is also available from the CLI:

```
gemstone-rs compare gemstone-py --batches
gemstone-rs compare gemstone-py --batches --json
gemstone-rs compare gemstone-py --totals
gemstone-rs compare all --totals
```

Batch	Work	Estimate
-	No active gemstone-rs parity batch remains	0 hours

Total: roughly **0 hours**. The Rust-backed native wheels have been published and verified from TestPyPI/PyPI for shared native-core integration.

Recent codegen batch status: closed. `codegen discover` now records protocol/source documentation, prefers source-header argument names, generated wrappers carry metadata tests, and `return=Symbol` generates Rust `String` helpers while preserving the GemStone domain type in config metadata.

Recent async/web batch status: closed. `SessionWorker` and `SessionWorkerPool` now expose awaitable calls, the Axum and Actix health handlers use that async path, and the `async_worker` example shows how Rust services can await GemStone work without moving `Session` across threads.

Recent shared-core batch status: closed. The `gemstone_rs::py_native` module gives `gemstone-py-native` plain Rust config, value, error, capability, and session wrappers, and `gemstone-py-native` now exposes an additive `RustCoreSession` bridge over that shared core while preserving existing Python return behavior. The installed CLI can still scaffold a starter PyO3 crate with `gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter`. It can also print the machine-readable migration checklist with `gemstone-rs py-native migration --json`, the compatibility shim map with `gemstone-rs py-native compatibility --json`, and the wrapper conformance target with `gemstone-rs py-native conformance --json`. It also prints the downstream handoff manifest with `gemstone-rs py-native handoff --json`, which bundles every artifact, schema, validation command, and acceptance check for the downstream `gemstone-py-native` wrapper. `gemstone-rs py-native publish-receipt --json` records the verified TestPyPI/PyPI workflow run ids, install commands, and package checks for the published Rust-backed wheels. `gemstone-rs py-native check-all` validates those checked-in fixtures together as the downstream shared-core gate. The

generated PyO3 starter exports those same reports through `gemstone_py_native.migration_json()`, `gemstone_py_native.compatibility_json()`, and `gemstone_py_native.conformance_json()`, plus `gemstone_py_native.handoff_json()`. The live GemStone smoke through the Rust-backed native path passed locally, and TestPyPI/PyPI install verification passed for the published Rust-backed wheels.

Current Gap Analysis

`gemstone-py` remains better when the user wants a batteries-included Python product surface: package extras, web adapters, async examples, a mature database explorer, and an examples launcher aimed at Python users and installed environments.

The most useful gemstone-rs catch-up work is to keep copying the product shape, not the implementation language:

- Installed discoverability: `gemstone-rs examples list` now mirrors the gemstone-py example map and gives VS Code structured JSON.
- No-live sanity check: `gemstone-rs hello` mirrors `gemstone-examples hello` so users can verify the CLI before configuring GemStone.
- Direct comparison: `gemstone-rs compare gemstone-py` turns this guide into a compact CLI summary with JSON output for tooling.
- Feature stream map: `gemstone-rs examples map` mirrors `gemstone-examples plan3-map`, but frames each stream in Rust crates, examples, docs, and gemstone-py reference points.
- Source-checkout launching: `gemstone-rs examples run <name>` now executes the matching Cargo example, with `--dry-run` for CI and documentation checks.
- Installed project scaffolding: `gemstone-rs examples scaffold quickstart`, `browser`, `bridge_root_mapping`, `derive_mapping`, `codegen_preview`, `codegen_workflow`, `codegen_discover`, `codegen_discover_mapping`, `profile_codegen_workflow`, `generated_wrapper_app`, `generated_mapping_app`, `http_service`, `axum_service`, and `actix_service` create standalone Cargo projects from the installed CLI, reducing the need for a source checkout. `profile_codegen_workflow` includes codegen config and project profile files, not only Rust source.
- Rust web service shape: `http_service` gives a real HTTP example with `/`,

`/health/local`, and `/health/gemstone` without adding framework dependencies. `gemstone-rs-axum` and `gemstone-rs-actix` now package the same route contract for framework users, and `examples/axum-service` plus `examples/actix-service` are checked crates using those adapters. They can start before credentials are configured and report `/health/gemstone` as a `503` JSON error until the pool is available. They also emit diagnostic adapter, route, request id, method, path, lifecycle, and handler-duration headers for route smoke tests and proxy logs. The checked Axum and Actix services also emit production-style service/cache/security middleware headers, and the smoke script asserts them. The installed CLI can scaffold `session_worker_pool`, `axum_service`, and `actix_service`. A required live route smoke mode now verifies `/health/gemstone` returns `{"result":7}` for both adapters when credentials are available. Broader framework coverage and deeper cancellation/shutdown tests remain future work.

- Editor workflow: `GemStone RS: Show Example Commands` exposes the same map in

  the Rust workbench and can run selected Cargo examples in a terminal.

- Remaining gap: gemstone-rs still needs installed templates for

  explorer-integrated workflows and richer generated wrapper profile variants, plus broader framework coverage, an async facade, and richer explorer screens before its onboarding feels as complete as gemstone-py.

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` now has an additive PyO3 bridge over the Rust core.
- `gemstone-rs examples scaffold py_native_pyo3_adapter` provides the starter

  wrapper shape for that migration.

- `gemstone-rs py-native migration --json` keeps wrapper status, live-smoke,

  and wheel-publish steps visible to CLI, CI, and VS Code.

- `gemstone-rs py-native conformance --json` keeps the expected PyO3

  module/session/shim surface fixture-backed for downstream wrapper CI.

- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on

  common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

gemstone-js vs gemstone-py

`gemstone-py` and `gemstone-js` solve the same broad problem from different runtime ecosystems. `gemstone-py` is the more mature Python path. `gemstone-js` is the async TypeScript path for JavaScript services, CLI tools, and npm-based delivery.

Use this guide when choosing between Python and TypeScript for GemStone/S application code. If the application already lives in Python, use `gemstone-py`. If the application already lives in Node, Deno, Bun, Express, Fastify, Fetch API, or Hono, `gemstone-js` is the natural fit once its native runtime path is proven for your platform.

Quick Answer

Question	Better default
Production Python app or script	`gemstone-py`
FastAPI, Litestar, or Django service	`gemstone-py`
TypeScript service or CLI	`gemstone-js`
Express, Fastify, Fetch API, or Hono service	`gemstone-js`
Most mature visual database explorer today	`gemstone-py`
npm package and TypeScript type workflow	`gemstone-js`
Lowest operational risk today	`gemstone-py`

Best Fit

Use case	gemstone-py	gemstone-js
Scripting	Pythonic, low-friction scripts and notebooks	Possible, but async JavaScript is less convenient for small one-off scripts
Web apps	FastAPI, Litestar, Django examples	Express, Fastify, Fetch API, Hono adapters
Async model	Python sync and async APIs	Async-first API throughout
Package ecosystem	PyPI/TestPyPI and Python packaging	npm package shape and optional `@gemstone-js/native`
Type system	Python type hints and generated access helpers	TypeScript signatures, manifests, decorators, and generated wrappers
Runtime support	Python interpreters plus optional native acceleration	Node native package, Deno/Bun FFI starters, mock runtime
Visual tooling	Python database explorer and VS Code workbench are more mature	Doctor, inspect, examples, benchmarks, migrations, but no equivalent visual explorer yet

Maturity Matrix

Capability	gemstone-py	gemstone-js
Install path	Mature PyPI path	Alpha npm package shape, `0.1.0-alpha.0` locally
Native bridge	Optional native acceleration already integrated into the Python release lane	Optional `@gemstone-js/native`, plus Deno/Bun FFI starters
Session API	Mature sync and async session APIs	Async-first `Session.connect()`, `execute()`, `perform()`, `performWith()`
Pooling	Python web/session patterns	Session pool with reset-aware release, warmup, health checks, and lifecycle events
Request scope	FastAPI/Litestar/Django examples	Shared `RequestScope` / `TransactionScope` across Express/Fastify/Fetch/Hono

Capability	gemstone-py	gemstone-js
Persistent roots	Supported	`PersistentRoot`, `GsDict`, `OrderedCollection`, `GStore`
Migrations	Supported in Python docs/examples	Module-style migrations CLI and metadata roots
Codegen	Python codegen and typed access demos	Manifest/decorator TypeScript codegen with schema validation
Benchmarks	Python performance docs and native checks	Benchmark report/baseline/compare/register tooling
Doctor/setup	Python setup docs and verification scripts	`gemstone-js-doctor` for runtime, env, native import, and optional live checks
Explorer/workbench	Stronger today	Not yet equivalent
Live CI confidence	Broader today	Needs more platform/runtime live coverage

Where gemstone-js Is Strong

- It is async-first, which matches modern JavaScript services.
- It has framework adapters for Express, Fastify, Fetch API, and Hono.
- It has a mock runtime, useful for TypeScript unit tests without a live stone.
- It has a rich package-tooling surface: doctor, examples, inspect, migrations, benchmarks, checksums, and API-contract checks.
- Its codegen model can use manifests, decorators, schemas, generated signatures, and TypeScript compile checks.
- It has high-level JavaScript value marshalling through `performWith()` and managed handles.

Where gemstone-py Is Still Ahead

- `gemstone-py` is more mature as a published, documented user path.
- The Python database explorer and VS Code workbench are richer product surfaces today.

- Python examples cover more end-to-end workflows with lower setup friction.
- The Python release lane has more established PyPI/TestPyPI/native wheel/VSIX verification history.
- Live GemStone coverage is broader around sync, async, framework, native, and lifetime behavior.

Recommended Work Batches for gemstone-js

Batch	Work	Estimate
1	Native publish confidence: npm package, `@gemstone-js/native`, clean install, live smoke	6-10 hours
2	Visual tooling: explorer/workbench for inspect, browse, codegen, and persistent roots	10-18 hours
3	Installed examples: quickstart, web adapters, migrations, codegen, persistence helpers	5-8 hours
4	Live CI: Node/Deno/Bun, framework adapters, pools, transactions, migrations	8-14 hours
5	Documentation/release polish: Medium article, PDF docs, release checklist, checksums	5-8 hours
6	Cross-project alignment: shared concepts with gemstone-py and gemstone-rs explorers/codegen	8-14 hours

Total: roughly **42-72 hours** to bring `gemstone-js` much closer to `gemstone-py` in maturity and product polish.

CLI Comparison

From the `gemstone-rs` checkout:

```
gemstone-rs compare gemstone-js
gemstone-rs compare gemstone-js --gaps
gemstone-rs compare gemstone-js --next
gemstone-rs compare gemstone-js --totals
gemstone-rs compare gemstone-js --batches
gemstone-rs compare gemstone-js --json
gemstone-rs compare gemstone-js --gaps --json
gemstone-rs compare gemstone-js --next --json
```

```
gemstone-rs compare gemstone-js --totals --json
gemstone-rs compare gemstone-js --batches --json
```

Use the JSON form when an editor, release script, or dashboard needs to render the archived TypeScript/Python comparison. `compare all` is intentionally scoped to the active gemstone-rs planning track now, so use the explicit `compare gemstone-js` commands above when you need this archived reference.

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

The design is connector-inspired, but not transparent persistence. That is an important Rust boundary. The mapping layer should make GemStone objects easier to use from Rust, while still making every remote read, write, transaction, and materialization decision visible in code review.

Which Mapping Layer Should I Use?

Layer	Use it when	What stays explicit
`Oop`	You need the raw GemStone object reference or are writing low-level tooling.	Selector sends, conversion, retain/release, and transaction policy.
`BridgeValue`	You are exploring a payload shape or building UI/CLI inspection tools.	Unsupported objects remain `BridgeValue::Oop`; depth limits are visible.
`BridgeMapped`	You have a stable dictionary-backed Rust payload.	Field names, key types, nested structs, vectors, maps, and option handling.
`#[derive(BridgeMapped)]`	You want normal Rust structs without writing the mapping boilerplate by hand.	Per-field key overrides and symbol/string key policy.
`BridgeRoot`		

Layer	Use it when	What stays explicit
	You need a stable GemStone dictionary root for Rust-owned payloads.	The root name, entry key type, commit/abort behavior, and write scope.
`Remote<T>` / `ObjectRef<T>`	You have an existing OOP and want an explicit cached Rust view of it.	`refresh(&mut Session)`, `set_value`, `save(&mut Session)`, and materialization profile.
Codegen wrappers	You want checked-in Rust wrappers around selectors or mapping configs.	Generated source is reviewed, diffed, checked, and committed.

For most application code, start with `BridgeRoot` plus `BridgeMapped`. Move to `Remote<T>` when the object identity itself matters, for example when a live GemStone object should be refreshed, edited as a Rust value, then explicitly saved back. Use raw `Oop` and `Session::perform` when the object is not yet stable enough to model.

Mapping Rules

The current policy is:

- no hidden network calls from normal Rust field access
- no automatic lazy loading from `Deref` or property access
- no automatic save on `Drop`
- all live operations require `&mut Session`
- all write persistence is explicit: `BridgeRoot::commit`, `BridgeRoot::transaction`, `Remote::save`, or `Session` transaction helpers
- direct `Oop` access remains available at every layer

That gives gemstone-rs a path toward MagLev-style productivity without making Rust code pretend a remote GemStone object is local memory.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::put_mapped_with_key_type`

- `BridgeRoot::put_field`
- `BridgeRoot::put_field_with_key_type`
- `BridgeRoot::put_string`
- `BridgeRoot::put_string_with_key_type`
- `BridgeRoot::put_symbol`
- `BridgeRoot::put_symbol_with_key_type`
- `BridgeRoot::put_smallint`
- `BridgeRoot::put_smallint_with_key_type`
- `BridgeRoot::put_bool`
- `BridgeRoot::put_bool_with_key_type`
- `BridgeRoot::put_vec`
- `BridgeRoot::put_vec_with_key_type`
- `BridgeRoot::put_map`
- `BridgeRoot::put_map_with_key_type`
- `BridgeRoot::put_optional`
- `BridgeRoot::put_optional_with_key_type`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_bridge_value`
- `BridgeRoot::get_bridge_value_with_key_type`
- `BridgeRoot::get_bridge_value_with_depth`
- `BridgeRoot::get_field`
- `BridgeRoot::get_field_with_key_type`
- `BridgeRoot::get_vec`
- `BridgeRoot::get_vec_with_key_type`
- `BridgeRoot::get_map`
- `BridgeRoot::get_map_with_key_type`
- `BridgeRoot::get_optional`
- `BridgeRoot::get_optional_with_key_type`
- `BridgeRoot::get_string`
- `BridgeRoot::get_string_with_key_type`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_smallint_with_key_type`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_bool_with_key_type`
- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_dictionary_with_key_type`
- `BridgeRoot::get_mapped`
- `BridgeRoot::get_mapped_with_key_type`
- `BridgeRoot::remove`
- `BridgeRoot::remove_with_key_type`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeRoot::keys`

- `BridgeRoot::contains_key`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_bridge_value`
- `BridgeDictionary::at_bridge_value_with_key_type`
- `BridgeDictionary::at_bridge_value_with_depth`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`
- `BridgeDictionary::at_bool`
- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_field`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeDictionary::at_map`
- `BridgeDictionary::at_optional`
- `BridgeDictionary::put_field`
- `BridgeDictionary::put_string`
- `BridgeDictionary::put_symbol`
- `BridgeDictionary::put_smallint`
- `BridgeDictionary::put_bool`
- `BridgeDictionary::put_vec`
- `BridgeDictionary::put_map`
- `BridgeDictionary::put_optional`
- `BridgeDictionary::contains_key`
- `BridgeDictionary::keys`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`
- `BridgeMapped`
- `#[derive(BridgeMapped)]`
- `BridgeFieldRead`
- `BridgeFieldWrite`
- `BridgeValue::Nil`
- `BridgeValue::Bool`
- `BridgeValue::SmallInt`
- `BridgeValue::String`
- `BridgeValue::Symbol`
- `BridgeValue::Oop`
- `BridgeValue::Dictionary`
- `BridgeValue::KeyedDictionary`
- `BridgeValue::Array`
- `BridgeValue::from_oop`
- `BridgeValue::from_oop_with_depth`
- `BridgeValue::shape_report`
- `BridgeValueShapeReport`

- `BridgeValueShapeNode`
- `Remote<T>` / `ObjectRef<T>`
- `MaterializationProfile::shallow`
- `MaterializationProfile::deep(max_depth)`
- `DictionaryKeyPolicy::Preserve`
- `DictionaryKeyPolicy::StringOnly`
- `ArrayMaterialization::Materialize`
- `ArrayMaterialization::Reject`
- `IdentityPolicy::PreserveOpaqueOops`
- `IdentityPolicy::ReportRepeatedOops`
- session-local OOP identity ids with `Session::identity_for_oop`

The default root is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict")?;
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
            labels: dictionary.at_map("labels")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
        labels: BTreeMap::from([("source".to_string(), "manual".to_string())]),
    };

    bridge_root.put_mapped("MyTestDict", &draft)?;
```

```
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.put_string("MyTestStatus", "ready")?;
    bridge_root.put_smallint("MyTestAmount", draft.amount)?;
    bridge_root.put_bool("MyTestApproved", true)?;
    bridge_root.put_vec("MyTestTags", &["priority".to_string(),
"demo".to_string()])?;
    bridge_root.put_optional("MyTestNote", &Some("front desk".to_string()))?;
    bridge_root.put_map("MyTestLabels", &draft.labels)?;

    assert_eq!(bridge_root.get_string("MyTestStatus")?, "ready");
    assert_eq!(bridge_root.get_smallint("MyTestAmount")?, draft.amount);
    assert!(bridge_root.get_bool("MyTestApproved")?);
    let tags: Vec<String> = bridge_root.get_vec("MyTestTags")?;
    assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
    let note: Option<String> = bridge_root.get_optional("MyTestNote")?;
    assert_eq!(note, Some("front desk".to_string()));
    let labels: BTreeMap<String, String> = bridge_root.get_map("MyTestLabels")?;
    assert_eq!(labels, draft.labels);

    bridge_root.put_map_with_key_type(
        "MyTestLabelsSymbol",
        BridgeKeyType::Symbol,
        &draft.labels,
    )?;
    let symbol_labels: BTreeMap<String, String> =
        bridge_root.get_map_with_key_type("MyTestLabelsSymbol",
BridgeKeyType::Symbol)?;
    assert_eq!(symbol_labels, draft.labels);

    bridge_root.commit()?;
    Ok(())
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
```

```
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}
```

Dictionary Mapping Patterns

Dictionary mapping shows up in three different places. Keep them separate:

Surface	What the key policy controls	Best API
BridgeRoot entry	the key used under `GemStoneRsBridgeRoot`	`put_with_key_type`, `get_*_with_key_type`
Dictionary value	the keys inside the stored GemStone `Dictionary`	`BridgeValue::dictionary`, `BridgeValue::keyed_dictionary`
Rust map field	string-keyed metadata or lookup values	`BTreeMap<String, T>`, `put_map`, `at_map`

Use `BTreeMap<String, T>` when the dictionary is JSON-like metadata. It stores and reads back string-keyed GemStone dictionary entries:

```
use gemstone_rs::{Config, Session};
use std::collections::BTreeMap;

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let labels = BTreeMap::from([
    ("channel".to_string(), "web".to_string()),
    ("priority".to_string(), "high".to_string()),
]);

bridge_root.put_map("BookingLabels", &labels)?;
let loaded: BTreeMap<String, String> = bridge_root.get_map("BookingLabels")?;
assert_eq!(loaded["channel"], "web");
```

Use `BridgeValue::keyed_dictionary` when the dictionary entries themselves must use Smalltalk symbols or a deliberate mix of string and symbol keys:

```
use gemstone_rs::{BridgeKey, BridgeKeyType, BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::keyed_dictionary([
    (BridgeKey::symbol("status"), BridgeValue::from("ready")),
    (BridgeKey::symbol("amount"), BridgeValue::from(100_i64)),
```

```
    (BridgeKey::string("externalId"), BridgeValue::from("web-42")),
]);

bridge_root.put("SmalltalkBooking", payload)?;

let dynamic = bridge_root.get_bridge_value("SmalltalkBooking")?;
assert!(matches!(dynamic, BridgeValue::KeyedDictionary(_)));

let mut dictionary = bridge_root.get_dictionary("SmalltalkBooking")?;
assert_eq!(
    dictionary.at_string_with_key_type("status", BridgeKeyType::Symbol)?,
    "ready"
);
assert_eq!(
    dictionary.at_smallint_with_key_type("amount", BridgeKeyType::Symbol)?,
    100
);
assert_eq!(dictionary.at_string("externalId")?, "web-42");
```

`BridgeValue::from_oop` reads a dictionary with only string keys as `BridgeValue::Dictionary`. If any key is a symbol, it preserves the per-entry policy as `BridgeValue::KeyedDictionary`. That makes explorer output and shape reports honest about what Smalltalk code will see.

One subtle point: `put_map_with_key_type("BookingLabels", BridgeKeyType::Symbol, &labels)` changes the key used to store `BookingLabels` in the containing dictionary. It does not make the entries inside `labels` symbol-keyed. To control entry keys inside the value, build a `BridgeValue::keyed_dictionary`.

Dynamic BridgeValue Inspection

When you do not want a typed struct yet, read a BridgeRoot value back as a dynamic `BridgeValue` tree:

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let payload = BridgeValue::dictionary([
        (
            "customer".to_string(),
            BridgeValue::dictionary([
                ("name".to_string(), BridgeValue::from("Tariq")),
                ("vip".to_string(), BridgeValue::from(true)),
            ]),
        ),
        (
            "items".to_string(),
```

```
            BridgeValue::array([
                BridgeValue::dictionary([
                    ("sku".to_string(), BridgeValue::from("A-1")),
                    ("quantity".to_string(), BridgeValue::from(2_i64)),
                ]),
            ]),
        ),
        ("state".to_string(), BridgeValue::Symbol("ready".to_string())),
        ("note".to_string(), BridgeValue::Nil),
    ]);

    bridge_root.put("BridgeValueInspection", payload.clone())?;
    let dynamic = bridge_root.get_bridge_value("BridgeValueInspection")?;
    assert_eq!(dynamic, payload);
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example bridge_value_inspection
```

Expected output includes:

```
dynamic BridgeValue: Dictionary({"customer": Dictionary(...), "items": Array(...),
"note": Nil, "state": Symbol("ready")})
bridge root identity: <number>
bridge root key count: <number>
```

`BridgeValue::from_oop_with_depth(session, oop, max_depth)` is also available for inspectors and tools. It reads nil, booleans, small integers, characters, strings, symbols, arrays, and string/symbol-keyed dictionaries. When the reader hits an unsupported object, a repeated object, or the depth limit, it returns `BridgeValue::Oop(oop)` instead of pretending the object is transparent Rust state.

For a compact relationship-oriented view, ask the value for a shape report:

```
let report = dynamic.shape_report();
println!("nodes: {}", report.total_nodes);
for node in report.nodes {
    println!("{} {} children={}", node.path, node.kind, node.child_count);
}
```

The CLI form is:

```
gemstone-rs bridge shape BookingDraft --depth 4
```

This reports paths such as `value.customer.#name`, `value.items[1].sku`, node kinds, key policy, child counts, opaque OOPs, report-local identity ids, repeated opaque references, and nil nodes. It is meant for relationship mapping review before generating typed wrappers.

When the same opaque OOP appears more than once, the report also includes an identity group. That gives the CLI, explorer, and VS Code webview a stable way to show relationship paths such as `value.items[2]` and `value.items[3]` both pointing at the same GemStone object.

You can turn the inspected shape into a starter codegen config before writing a typed struct by hand:

```
use gemstone_rs::{codegen, BridgeValue};

let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
]);

let config = codegen::mapping_config_from_bridge_value("BookingDraft", &payload);
println!("{config}");
```

Run the offline example:

```
cargo run -p gemstone-rs --example bridge_mapping_preview
```

Or infer from a live BridgeRoot value:

```
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
```

The output is intentionally reviewable text, not automatic persistence magic. Opaque OOPs, nil fields, symbols, empty arrays, and mixed arrays receive `doc=` notes so you can choose a narrower type before running codegen.

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}
```

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        amount: 100,
        customer: CustomerDraft {
            name: "Tariq".to_string(),
        },
        tags: vec!["priority".to_string(), "demo".to_string()],
        labels: BTreeMap::from([("source".to_string(), "derive".to_string())]),
        note: None,
    };

    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }
bridge root identity: <number>
```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
`String`	`String`	`BridgeFieldRead` / `at_string`
`i64`	`SmallInteger`	`BridgeFieldRead` / `at_smallint`
`bool`	`true` / `false`	`BridgeFieldRead` / `at_bool`
`Oop`	raw object reference	`BridgeFieldRead` / `at_oop`
another `BridgeMapped` struct	nested `Dictionary`	`BridgeDictionary::at_mapped`
`Vec<T>`	`Array`	`BridgeDictionary::at_vec`
`BTreeMap<String, T>`	string-keyed `Dictionary`	`BridgeDictionary::at_map`
`Option<T>`	value, `nil`, or missing key	`BridgeFieldRead`

This lets a Rust payload keep normal nested shape:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

Optional fields are useful when a BridgeRoot dictionary evolves over time. `None` writes as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`. `Some(value)` reads through the inner field type, so `Option<CustomerDraft>` still reports nested mapping errors with the full field path.

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234
field booking.customer.name expected GemStone value type String, got OOP 1234
field tags[2] expected GemStone value type String, got OOP 1234
field labels["source"] expected GemStone value type String, got OOP 1234
```

Nested mapped read-back preserves the full field path, and array read-back reports the failing element index. Both are important when generated mappings read back nested payloads from a live stone. Lookup failures are also wrapped with the current path, so missing keys and invalid nested arrays point at `booking.items` or `booking.items[2]` instead of only returning a generic GemStone lookup error.

Dynamic `BridgeValue` read-back uses the same path-aware machinery when called through `BridgeRoot::get_bridge_value` or `BridgeDictionary::at_bridge_value`. That makes it useful for explorer panels and generated-wrapper debugging before you settle on a typed `BridgeMapped` struct.

Key Policy

GemStone dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;

bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored")?;
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

The typed helpers have the same key-policy variants:

```
bridge_root.put_field_with_key_type("BookingLabels", BridgeKeyType::Symbol,
&draft.labels)?;
let labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("BookingLabels", BridgeKeyType::Symbol)?;
```

Those variants choose the lookup key in the containing dictionary. For symbol-keyed entries inside the dictionary value itself, use the dictionary mapping pattern above.

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Remote Object Handles

`Remote<T>` is the Rust-native proxy layer. It is deliberately explicit:

- it stores an `Oop`
- it stores type metadata such as `UserGlobals:BookingDraft`
- normal field access never talks to GemStone
- `refresh(&mut Session)` reads the GemStone object into a cached Rust value

- `set_value(value)` marks the cached value dirty
- `save(&mut Session)` writes the cached mapped value back to the same GemStone dictionary object

```
use gemstone_rs::{BridgeMapped, Config, MaterializationProfile, Remote, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    status: String,
    amount: i64,
    labels: BTreeMap<String, String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let initial = BookingDraft {
        status: "draft".to_string(),
        amount: 100,
        labels: BTreeMap::from([("source".to_string(), "remote-
example".to_string())]),
    };

    let oop = {
        let mut bridge_root = session.bridge_root()?;
        bridge_root.put_mapped("RemoteBookingDraft", &initial)?;
        bridge_root.get_oop("RemoteBookingDraft")?
    };

    let mut remote = Remote::<BookingDraft>::with_type(oop,
"UserGlobals:BookingDraft")
        .with_profile(MaterializationProfile::deep(4));

    let loaded = remote.refresh(&mut session)?.clone();
    assert_eq!(loaded, initial);

    let mut updated = loaded;
    updated.status = "confirmed".to_string();
    remote.set_value(updated);
    assert!(remote.is_dirty());
    remote.save(&mut session)?;
    assert!(!remote.is_dirty());

    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example remote_object_mapping
```

Expected output includes:

```
remote loaded: BookingDraft { status: "draft", amount: 100, labels: {"source":
"remote-example"} }
remote saved: BookingDraft { status: "confirmed", amount: 100, labels: {"source":
"remote-example"} }
```

`ObjectRef<T>` is an alias for `Remote<T>` when that name reads better in application code.

This is not transparent persistence. The handle does not save on drop, does not fetch fields lazily, and does not hide network I/O behind normal Rust access. Every GemStone operation still takes `&mut Session`.

Materialization Profiles

Materialization profiles describe how far a remote value should be read and how strictly the resulting `BridgeValue` tree should be validated:

```
use gemstone_rs::{
    ArrayMaterialization, DictionaryKeyPolicy, IdentityPolicy,
MaterializationProfile,
};

let profile = MaterializationProfile::deep(4)
    .with_dictionary_key_policy(DictionaryKeyPolicy::Preserve)
    .with_array_materialization(ArrayMaterialization::Materialize)
    .with_identity_policy(IdentityPolicy::ReportRepeatedOops);
```

Useful profiles:

Profile	Behavior
`MaterializationProfile::shallow()`	Keep non-immediate remote objects opaque.
`MaterializationProfile::deep(4)`	Read supported dictionaries and arrays up to depth 4.
`DictionaryKeyPolicy::StringOnly`	Reject symbol-keyed dictionaries when a JSON-like shape is required.
`ArrayMaterialization::Reject`	Fail fast if a mapping path unexpectedly contains an array.
`IdentityPolicy::ReportRepeatedOops`	Keep shape reports useful for repeated opaque OOP references.

The profile feeds `Remote<T>::materialize`, `Remote<T>::shape_report`, and tool surfaces such as the explorer and VS Code webview. It does not make remote objects transparent; it makes the read policy explicit and reviewable.

Connector-Style Config

For BridgeRoot dictionaries, `field = ... | type=... | key=...` is enough. For remote GemStone classes, codegen configs can also carry connector-style selector metadata:

```
mapped = Booking
class = UserGlobals:OkzBooking
field = Booking.status | selector=status | return=Symbol
field = Booking.customer | selector=customer | return=Mapped<Customer>
```

For a concrete checked-in example, see:

```
examples/codegen/connector-mapping.codegen
```

It combines `class = UserGlobals:OkzBooking`, selector metadata, return metadata, relationship fields, and dictionary fallback keys. Run `gemstone-rs codegen explain examples/codegen/connector-mapping.codegen` to see the mapping summary without writing generated files.

This lets one config describe both sides of the bridge:

- `class = UserGlobals:OkzBooking` records the GemStone class to inspect or wrap
- `selector=status` records how the remote object is read
- `return=Symbol` records the Smalltalk-facing return type
- `return=Mapped<Customer>` records a relationship to another mapped Rust type

The initial implementation parses and reports this connector metadata through `codegen explain` and `codegen explain --json`. Generation still stays conservative: dictionary-backed `BridgeMapped` code is emitted as before, while remote selector wrappers can be layered on top in a later generator pass.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("BookingDraft")?;
    assert_eq!(loaded, draft);
```

```
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft")?;
let identity = bridge_root.identity_id();
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Relationship-Shaped Payloads

Use nested structs and vectors when the Rust boundary needs a relationship-like shape but you still want explicit persistence. This keeps the GemStone side a plain dictionary graph and keeps the Rust side strongly typed:

```
use gemstone_rs::BridgeMapped;
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    name: String,
    email: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct LineItemDraft {
    sku: String,
    quantity: i64,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    items: Vec<LineItemDraft>,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

A failed nested read reports the path that a user would naturally inspect:

```
booking.items[2].customer.name expected GemStone value type String, got OOP 1234
```

This is the practical middle ground between raw OOPs and transparent object persistence: relationships are visible in config and generated Rust source, and the low-level OOP API remains available for special cases.

Comparison With MagLev/GBS

The MagLev branch example uses:

```
session bridgeRoot at: #MyTestDict put: payload.
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels
field = BookingDraft.note | type=Option<String> | key=note
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

For quick CLI inspection of the bridge root itself:

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge value BookingDraft --depth 4
gemstone-rs bridge shape BookingDraft --depth 4
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
gemstone-rs bridge sample-config BookingDraft
```

Use `--root OtherBridgeRoot` when you intentionally use a non-default root dictionary, and `--key-type String|Symbol` when you want the key policy to be spelled out in scripts. Equals-style options such as `--root=OtherBridgeRoot`, `--key-type=Symbol`, and `--type=SmallInt` are accepted for CI scripts. The generic `bridge put --type String|Symbol|SmallInt|Bool` form is still available when the value type comes from script data.

`bridge keys` lists each root key with its key OOP, class OOP, `printString`, and session-local identity id.

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
```

```
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The write endpoints require `gemstone-rs-explorer --allow-write`. Without that flag they return HTTP 403, matching the explorer's read-only default.

The VS Code workbench adds the same object-mapping workflow:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oop` remains visible and available.
- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain --json examples/codegen/gemstone-
rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

Method Metadata

Option	Example	Purpose
`args`	`args=id,user`	

Option	Example	Purpose
		Names generated Rust function arguments.
`args` with types	`args=id:SmallInt,name:String,selector:Symbol,enabled:Bool`	Generates native Rust parameters and converts them to GemStone OOPs before `perform`.
`return`	`return=String` or `return=Symbol`	Generates a typed return helper instead of `Value`.
`doc`	`doc=Find by id.`	Writes a Rust doc comment above the generated method.

When `args` is omitted, codegen infers useful argument names from selector keywords. For example `at:put:` becomes `at, put`, and `withCustomer:amount:` becomes `customer, amount`. Live discovery goes one step further: when method source is available, it prefers the Smalltalk source header, so `at: anIndex put: aValue` becomes `an_index, a_value`. Provide explicit `args=...` when you want a different Rust API name.

Untyped arguments stay explicit as `Oop`. Add a type after the argument name when the generated wrapper should accept native Rust values:

```
method = UserGlobals:OkzBooking class>>findById: | args=id:SmallInt | return=Oop
method = Object>>perform: | args=selector:Symbol
method = UserGlobals:Order>>statusSymbol | return=Symbol
method = UserGlobals:User>>named:active: | args=name:String,active:Bool | return=Oop
```

Those examples generate signatures like:

```
pub fn find_by_id(&mut self, id: i64) -> Result<Oop> {
    let id = self.session.smallint_oop(id);
    let value = self.session.perform(self.oop, "findById:", &[id])?;
    // typed return conversion follows
}
```

```
pub fn perform(&mut self, selector: impl AsRef<str>) -> Result<Value> {
    let selector = self.session.new_symbol(selector.as_ref())?;
    self.session.perform(self.oop, "perform:", &[selector])
}
```

Generated wrapper files include `#[cfg(test)]` stubs for stable surface names, method metadata, and mapped-field metadata. That gives downstream crates a cheap compile-time smoke check for wrapper names, selectors, argument counts, typed returns, BridgeRoot field keys, key policies, and field types. Use `codegen explain` before generating when you want a readable summary of the output file, test stubs, wrapper classes, selectors, argument names and types, return helpers, mapped structs, and BridgeRoot field mappings:

```
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
```

The JSON form is intended for editor and explorer integrations that want to render the output path, generated test stubs, class wrappers, selector arguments, argument types, return helpers, and mapped fields as structured data. Method entries include both the legacy `args` name list and an `arguments` array with `{name, type, rustType}` objects for richer UI rendering. The `testStubs` array now reports all generated tests so VS Code, CI, and the explorer can show exactly what wrapper invariants are covered.

Machine-readable schema files are committed for tooling:

```
schemas/gemstone-rs.codegen.schema.json
schemas/gemstone-rs.codegen-explain.schema.json
schemas/gemstone-rs.codegen-profiles.schema.json
schemas/gemstone-rs.profile-check.schema.json
schemas/gemstone-rs.py-native.schema.json
schemas/gemstone-rs.py-native-samples.schema.json
schemas/gemstone-rs.py-native-smoke.schema.json
schemas/gemstone-rs.py-native-migration.schema.json
schemas/gemstone-rs.py-native-compat.schema.json
schemas/gemstone-rs.py-native-conformance.schema.json
schemas/gemstone-rs.py-native-handoff.schema.json
schemas/gemstone-rs.py-native-check-all.schema.json
```

`gemstone-rs.codegen` remains the line-oriented CLI format. The config schema describes an equivalent structured JSON model for editor panels and generated summaries, while the explain schema matches `codegen explain --json`. The py-native schemas match `gemstone-rs py-native capabilities --json`, `samples --json`, `smoke --dry-run --json`, `migration --json`, `compatibility --json`, `conformance --json`, and `handoff --json`. Use `gemstone-rs py-native check`, `check-samples`, `check-smoke`, `check-compat`, `check-conformance`, and `check-handoff` to compare checked-in fixtures against those shared

core renderers without linking against internal CLI code. Use `gemstone-rs py-native check-all --json` when downstream `gemstone-py-native` CI wants one schema-backed gate for every checked-in fixture. The profile check schema matches `profile check --json` and the explorer `/api/codegen/profiles/check` endpoint.

Supported return types:

Config value	Rust return
`Value`	`gemstone_rs::Value`
`String`	`String`
`Symbol`	`String`
`SmallInt`	`i64`
`Bool`	`bool`
`Oop`	`gemstone_rs::Oop`

Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot`:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels |
doc=String-keyed labels.
field = BookingDraft.note | type=Option<String> | key=note | doc=Optional note.
```

Supported field types are `String`, `SmallInt`, `Bool`, `Oop`, `Mapped<OtherStruct>`, `Vec<T>`, `BTreeMap<String, T>`, and `Option<T>`. `Map<String, T>` is accepted as a shorter alias, and `Dictionary<T>` is an alias for `BTreeMap<String, T>`. Map fields store string-keyed relationship metadata as a GemStone `Dictionary`. Optional fields write `None` as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`.

Field options:

Option	Example	Purpose
`type`	`type=Option<String>`	Rust/GemStone field conversion.

Option	Example	Purpose
`key`	`key=amount`	GemStone dictionary key.
`key_type`	`key_type=Symbol`	Use string keys or symbol keys explicitly.
`selector`	`selector=status`	Smalltalk selector used by a connector-style remote field.
`return`	`return=Symbol`	Smalltalk-facing remote return type; also accepts `Mapped<Customer>`.
`doc`	`doc=Booking amount.`	Writes a Rust doc comment above the field.

Connector-style field metadata can live beside dictionary-backed mapping:

```
mapped = Booking
class = UserGlobals:OkzBooking
field = Booking.status | selector=status | return=Symbol
field = Booking.customer | selector=customer | return=Mapped<Customer>
```

The committed sample `examples/codegen/connector-mapping.codegen` shows a fuller remote-object mapping with selector metadata, return metadata, dictionary-backed fallback keys, and relationship fields:

```
gemstone-rs codegen explain examples/codegen/connector-mapping.codegen
gemstone-rs codegen explain --json examples/codegen/connector-mapping.codegen
```

`codegen explain` and `codegen explain --json` report this metadata so the explorer and VS Code can show the remote selector mapping. Generated `BridgeMapped` structs still use explicit dictionary fields; selector-backed remote wrapper generation is intentionally a later pass.

Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

Live method discovery is deliberately conservative. It records selectors, prefers argument names from the source header, falls back to keyword selector names, keeps argument types as explicit `Oop` until a user narrows them, and adds protocol/source context to `doc=...` when the browser can fetch it. That gives generated wrappers stable Rust argument names without pretending to know return or argument types that GemStone/S has not declared.

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

From an installed CLI, scaffold standalone projects:

```
gemstone-rs examples scaffold codegen_discover ./gemstone-rs-codegen-discover
gemstone-rs examples scaffold codegen_discover_mapping ./gemstone-rs-codegen-
discover-mapping
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
```

`profile_codegen_workflow` writes `gemstone-rs.codegen` and `gemstone-rs.codegen-profiles.json` into the generated project, then the Rust program runs explain, generate, and profile check against those files.

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

Explorer Profiles

The local explorer can save repeatable codegen workflows as profiles. A profile captures:

- codegen root
- config path
- mapped Rust type
- GemStone class

Use the browser UI to save a local profile, export a single profile as JSON, or load a project-level profile file. The repository includes a sample:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Validate it from CI or a terminal:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen preview-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen diff-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
gemstone-rs codegen generate-profile default gemstone-rs.codegen-profiles.json
```

See Codegen Profile Schema and `schemas/gemstone-rs.codegen-profiles.schema.json` for editor validation.

The project profile file uses this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Start the explorer with a project root:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Profile writes are disabled unless the explorer was started with `--allow-write`. The server rejects path traversal in `config=` and `profile_file=` after URL decoding, rejects traversal in `root=`, keeps writes under the configured codegen root by default, and requires `--allow-absolute-write-paths` before absolute write targets are accepted. Project profile saves reject unsupported top-level fields, unsupported profile fields, missing or duplicate profile names, and non-string profile values.

Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```
#[path = "../../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}
```

Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
    /// String-keyed labels.
    pub labels: BTreeMap<String, String>,
    /// Optional note.
    pub note: Option<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles
- Save Project Profiles
- Export Codegen Profile
- Validate Project Profiles
- Check Project Profiles
- Open Codegen Docs

`Generate Wrappers` shows the diff first and asks before writing when output would change.

Codegen Profile Schema

`gemstone-rs` project profile files make explorer and VS Code codegen workflows repeatable. The default file name is:

```
gemstone-rs.codegen-profiles.json
```

This repository includes a sample:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

The JSON Schema is checked in at:

```
schemas/gemstone-rs.codegen-profiles.schema.json
```

Validate the sample from a checkout:

```
cargo run -p gemstone-rs-cli -- profile sample
cargo run -p gemstone-rs-cli -- profile init gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
```

```
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile list examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile show default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile resolve default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile check --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

For an installed CLI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
```

Expected output:

```
profile ok: examples/codegen/gemstone-rs.codegen-profiles.json (4 profiles: default,
object-wrapper, bridge-mapping, connector-mapping)
profile check: examples/codegen/gemstone-rs.codegen-profiles.json (4 profiles)
summary: 4 ok, 0 stale, 0 errors, 4 total
ok      default config=examples/codegen/gemstone-rs.codegen     output=examples/
codegen/generated/gemstone_wrappers.rs
```

The JSON form includes aggregate fields so CI and editor integrations can fail with a concise summary:

```
{"success":true,"ok":true,"profileCount":4,"okCount":4,"staleCount":0,"errorCount":0}
```

The profile check JSON shape is shared by the CLI, explorer, and VS Code Workbench. Its schema lives at:

```
schemas/gemstone-rs.profile-check.schema.json
```

Shape

```
{
  "kind": "gemstone-rs-explorer-codegen-profiles",
  "version": 1,
  "profiles": [
    {
      "name": "default",
      "config": "examples/codegen/gemstone-rs.codegen",
      "root": "",
      "mapped": "BookingDraft",
      "className": "Object"
    }
  ]
}
```

Top-level fields:

- `kind` must be `gemstone-rs-explorer-codegen-profiles`.
- `version` must be `1`.
- `profiles` must be an array.

Profile fields:

- `name` is required, string-valued, non-empty, and unique.
- `config` is optional and string-valued.
- `root` is optional and string-valued.
- `mapped` is optional and string-valued.
- `className` is optional and string-valued.

Unknown top-level or profile fields are rejected. Non-string profile fields are rejected.

VS Code

Use `GemStone RS: Show Sample Project Profiles` to open the built-in sample in an untitled JSON editor, `GemStone RS: Create Project Profiles` to write the sample to a project file, and `GemStone RS: Validate Project Profiles` to run the same CLI validator and show the result in the GemStone RS output panel. `GemStone RS: List Project Profiles`, `GemStone RS: Show Project Profile`, and `GemStone RS: Resolve Project Profile` render parsed profile details through the same CLI parser, so you can check the active config/root/mapped/class fields and resolved config path without opening the explorer. Profile-specific commands use a

QuickPick populated from the profile file. The workbench also contributes JSON validation for files named `gemstone-rs.codegen-profiles.json`, using the packaged schema copy in `vscode-gemstone-rs-workbench/schemas/`.

Without the extension, associate the schema with the profile file in VS Code:

```
{
  "json.schemas": [
    {
      "fileMatch": ["gemstone-rs.codegen-profiles.json"],
      "url": "./schemas/gemstone-rs.codegen-profiles.schema.json"
    }
  ]
}
```

Explorer Safety

The explorer validates project profile JSON before saving. It also rejects write paths containing `..` after URL decoding. Relative profile writes stay inside the configured `--codegen-root`; absolute write targets require `--allow-absolute-write-paths`.

Explorer Guide

`gemstone-rs-explorer` is a local-only HTTP explorer built on the Rust API. It is useful for proving the browser, inspect, eval, and codegen surfaces before embedding richer tooling in VS Code or another frontend.

Install and Run

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

From a checkout:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

If you launch the explorer outside the project checkout, point codegen discovery and relative config paths at the checkout explicitly:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Open:

```
http://127.0.0.1:8787/
```

The home page is a small browser UI over the same JSON endpoints. It can:

- browse dictionaries, classes, protocols, methods, and source
- show the current browse path, class-side toggle, result count, and selected method source in the detail pane
- run a setup assistant that checks the env file, live configuration, GCI library, codegen config, and project profiles
- run doctor/status checks
- compare gemstone-rs with gemstone-py and show the remaining batch/hour estimate without opening the terminal
- inspect BridgeRoot, list keys, and render value payload summaries
- run codegen sample/discover/preview/diff/check/generate from an editable config path
- refresh a picker of known `.codegen` files and load one without typing the path by hand
- set a project codegen root through `--codegen-root`, or override it from the page before refreshing configs
- reuse recently loaded, saved, previewed, diffed, checked, or generated config paths from local browser storage
- save named codegen profiles containing config root, config path, mapped Rust type, and GemStone class
- export and import codegen profiles as versioned JSON for sharing between browsers or teammates
- load project profile files from the codegen root, and save them back when the explorer runs with `--allow-write`

- load and save the selected codegen config file; saving is still gated by

  `--allow-write`, accepts a POST body for larger configs, and validates the config before writing

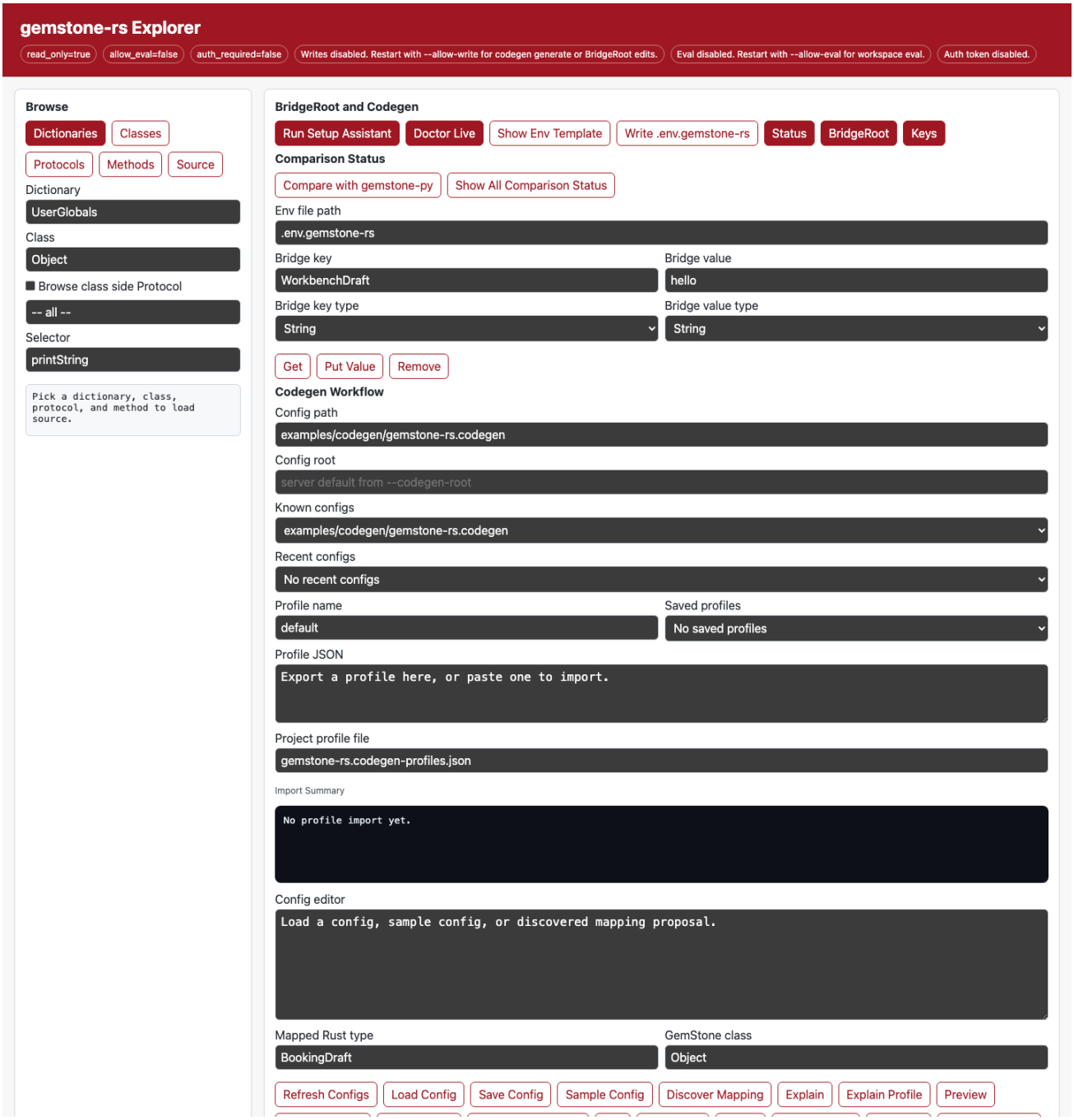
- render generated wrapper source, generated mapping config, colored unified

  diff output, current generated output files, and a side-by-side diff in a dedicated detail pane

- remember the current browser fields locally across reloads
- put/remove BridgeRoot values with explicit string/symbol key policy and

  string/small-int/bool value policy when `--allow-write` is enabled

Screenshot:



Refresh it with:

```
make screenshots
```

See Screenshot Workflow for the repeatable capture process.

Safety Defaults

The explorer:

- binds to `127.0.0.1` by default
- rejects non-loopback hosts
- starts read-only
- requires `--allow-eval` before workspace evaluation
- requires `--allow-write` before codegen writes
- supports an optional local auth token with `--auth-token` or `--auth-token-env`
- reads GemStone credentials from the same `GS_*` environment as the CLI

Do not expose it publicly. The token option is for local defense in depth on a loopback machine; it is not a substitute for TLS, user accounts, or a production auth layer.

To require a token for all explorer pages and APIs except `/health`:

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
```

Then use either a query token for browser workflows:

```
open 'http://127.0.0.1:8787/?token=replace-with-a-local-random-token'
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

Or a header for scripts:

```
curl -s \
  -H 'X-GemStone-RS-Token: replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/config
curl -s \
  -H 'Authorization: Bearer replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/status
```

Browse Endpoints

Run these from a second terminal:

```
curl -s http://127.0.0.1:8787/health
curl -s http://127.0.0.1:8787/api/config
curl -s 'http://127.0.0.1:8787/api/setup/assistant?config=examples/codegen/gemstone-
rs.codegen&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
```

```
curl -s http://127.0.0.1:8787/api/doctor
curl -s 'http://127.0.0.1:8787/api/doctor?live=1'
curl -s http://127.0.0.1:8787/api/status
curl -s http://127.0.0.1:8787/api/browse/dictionaries
curl -s 'http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals'
curl -s 'http://127.0.0.1:8787/api/browse/protocols?class=Object&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--
&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/source?
class=Object&selector=printString&meta=0'
curl -s 'http://127.0.0.1:8787/api/inspect?oop=20'
```

Expected shapes:

```
{"status":"ok"}
{"host":"127.0.0.1","port":8787,"readOnly":true,"allowEval":false}
{"success":true,"environment":{"GS_PASSWORD":{"status":"set","masked":true}}}
{"connected":true,"sessionId":12345,"needsCommit":false,"inTransaction":false}
{"success":true,"dictionaries":["UserGlobals"]}
```

Eval

Eval is disabled by default:

```
gemstone-rs-explorer --allow-eval
```

Then:

```
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Expected:

```
{"type":"smallInt","value":7}
```

Codegen Endpoints

The setup actions on the home page can show the same safe `GS_*` environment template as `gemstone-rs env sample`, or write `.env.gemstone-rs` when the explorer was started with `--allow-write`. Start the explorer with `--env-file .env.gemstone-rs` to use that file for all live browse, inspect, BridgeRoot, and codegen discovery actions. `Run Setup Assistant` checks the env file, GemStone configuration, GCI library, selected codegen config, and project profile file in one response.

The home page includes a Codegen Workflow panel. Set `Config path`, optionally set `Config root`, or start the server with `--codegen-root`. `Refresh Configs` and the `Known configs` picker choose an existing `.codegen` file relative to that root. The `Recent configs` picker remembers paths used by Load, Save, Preview, Diff, Check, and Generate in local browser storage. `Saved profiles` store the root, config path, mapped Rust type, and GemStone class as a named local browser profile. `Export Profile` writes a versioned JSON payload to `Profile JSON`; paste that JSON into another explorer and use `Import Profile` to merge it into saved profiles. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the codegen root by default, while `Save Project Profiles` writes the current saved profiles back to that file when the explorer runs with `--allow-write`. Optionally enter a mapped Rust type and GemStone class, then use:

- `Config root` to override the server default for config discovery and

  relative config paths

- `Refresh Configs` to scan the project root for known `.codegen` files
- `Recent configs` to return to the last few config paths used in this browser
- `Save Profile` and `Saved profiles` to switch between named codegen workflows
- `Export Profile` and `Import Profile` to share a codegen workflow as JSON
- `Load Project Profiles` and `Save Project Profiles` to use a committed

  project-level profile file; the server rejects invalid profile schemas and the browser reports which profiles are new, replaced, or unchanged

- `Check Project Profiles` to verify every profile in the project profile file

  and show ok/stale/error counts in one report; the detail pane renders a table with profile name, config path, output file, freshness, and per-profile Preview, Diff, Check, and Generate actions

- `Load Config` to read the selected config file into the editor
- `Save Config` to POST the editor contents, validate them, and write the file

  when the explorer was started with `--allow-write`

- `Sample Config` to view the starter config format
- `Discover Mapping` to ask a live stone for a mapping config proposal
- `Explain` to render a structured codegen summary, including output path,

  generated test stubs, wrappers, return helpers, and mapped fields

- `Explain Profile` to render the same structured summary after resolving a

  named profile from the project profile file

- `Preview` to inspect generated Rust wrappers without writing files
- `Preview Profile` to inspect generated wrappers after resolving a named

  project profile

- `Read Output` to load the currently committed generated output file named by the config, without regenerating it
- `Read Profile Output` to load the generated output file after resolving a named project profile
- `Diff` to compare generated output with the committed file; the detail pane shows both the exact unified diff and a side-by-side view for review
- `Diff Profile` to compare generated output from a named project profile
- `Check` to fail when generated output is stale
- `Check Profile` to run the stale-output check from a named project profile
- `Generate` to write wrappers when the explorer was started with `--allow-write`
- `Generate Profile` to resolve a named project profile and write its wrappers when the explorer was started with `--allow-write`

The VS Code webview wraps the same page with native handoff actions. It can run live browse probes from the side inspector, open method source in a VS Code editor, open the configured codegen/profile files, open the generated output file reported by codegen explain/check/profile status, render generated wrapper source in an editable webview textarea, open that text as an untitled VS Code draft, and save edited generated output back to the configured output path after a confirmation prompt. The save path is constrained to the configured checkout or workspace. The webview also offers Launch Explorer/Open Browser/Copy URL fallbacks when the loopback explorer is not running.

The `Comparison Status` buttons call read-only local endpoints:

- `Compare with gemstone-py` renders the same short answer as `gemstone-rs compare gemstone-py --status`
- `Show All Comparison Status` renders the active gemstone-rs batch count, currently **0 batches** and roughly **0-0 hours**

Read-only endpoints:

```
curl -s http://127.0.0.1:8787/api/compare/gemstone-py/status
curl -s http://127.0.0.1:8787/api/compare/all/status
curl -s http://127.0.0.1:8787/api/codegen/sample
curl -s 'http://127.0.0.1:8787/api/codegen/configs?root=.'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles?profile_file=gemstone-rs.codegen-
profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles/check?profile_file=examples/
```

```
codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/config?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/preview-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/diff-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/output?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/output-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/check-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The repository includes a sample project profile file:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Expected read-only results include the config list and a current generated output check:

```
{"success":true,"root":".","configs":["examples/codegen/gemstone-rs.codegen"]}
{"success":true,"exists":true,"upToDate":true}
{"success":true,"ok":true,"profileCount":4,"okCount":4,"staleCount":0,"errorCount":0}
```

Write-gated endpoint:

```
gemstone-rs-explorer --allow-write
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/generate?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/generate-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s -X POST \
  'http://127.0.0.1:8787/api/env/write?path=.env.gemstone-rs'
curl -s -X POST \
  --data-binary @gemstone-rs.codegen-profiles.json \
  'http://127.0.0.1:8787/api/codegen/profiles/save?profile_file=gemstone-rs.codegen-
```

```
profiles.json'
curl -s -X POST \
  --data-binary @examples/codegen/gemstone-rs.codegen \
  'http://127.0.0.1:8787/api/codegen/config/save?config=examples/codegen/
draft.codegen'
```

Expected:

```
{"success":true,"output":"examples/codegen/generated/
gemstone_wrappers.rs","bytes":1234}
{"success":true,"config":"examples/codegen/draft.codegen","bytes":512}
```

Project profile files use this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Validate the file before sharing or saving it from CI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
```

The schema reference is Codegen Profile Schema.

For write endpoints, relative `config=` and `profile_file=` paths are resolved inside the codegen root. Paths containing `..` are rejected after URL decoding, `root=...` traversal is rejected as well, and absolute write paths require starting the explorer with `--allow-absolute-write-paths`.

The browser UI also asks for confirmation before BridgeRoot writes, env-file writes, profile saves, config saves, and codegen generate actions. Server-side guards still decide the real access policy: write endpoints return `403` unless the explorer was started with `--allow-write`.

Project profile saves validate the JSON before writing. The top-level object must contain only `kind`, `version`, and `profiles`; profile names are required and unique; and each profile may contain only string-valued `name`, `config`, `root`, `mapped`, and `className` fields.

BridgeRoot Endpoints

BridgeRoot inspection and mapping-config preview use the same live session configuration as the browser endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&key_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/shape?key=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-preview?
key=BookingDraft&mapped=BookingDraft&depth=4'
```

Use these endpoints as a deliberate mapping workflow:

1. Inspect the live value with `/api/bridge/get`.
2. Review the relationship paths and repeated identities with

   `/api/bridge/shape`.

3. Create a first mapping config with `/api/bridge/mapping-preview`.
4. Review the generated config before writing files or committing codegen

   output.

The browser UI exposes `Bridge key type` and `Bridge value type` controls so you can explicitly test string-key, symbol-key, string-value, symbol-value, small-int-value, and bool-value mappings before encoding them in a reusable config file. `BridgeRoot Value` now shows both the raw OOP/class/printString summary and a nested `BridgeValue` tree for string/symbol-keyed dictionaries and arrays. `Preview Mapping Config` reads the same nested value and writes a starter `mapped = ...` / `field = ...` config into the detail pane, including review notes for opaque OOPs, nil fields, symbols, empty arrays, and mixed arrays. `Shape Report` turns the value into a relationship table with paths, node kinds, key policy, child counts, nil counts, opaque OOP counts, report-local identity ids, and repeated opaque references. It also groups repeated opaque OOPs so the UI can show every path pointing at the same GemStone object. The page persists the selected key, value, class, selector, config, and mapping fields in browser local storage; use `Clear Saved Fields` to reset the page back to the documented defaults.

The explorer does not turn GemStone objects into transparent local state. Object identity stays visible as OOPs, unsupported values remain opaque in the `BridgeValue` tree, and writes remain behind the `--allow-write` gate. Rust-side `Remote<T>` handles use the same explicit model: inspect first, materialize with a profile, then call `refresh` or `save` with `&mut Session`.

The same nested read-back is available from the CLI:

```
gemstone-rs bridge value BookingDraft --depth 4
gemstone-rs bridge shape BookingDraft --depth 4
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
```

Writes are disabled unless the explorer is started with `--allow-write`:

```
gemstone-rs-explorer --allow-write
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchState&value=ready&value_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchCount&value=7&value_type=SmallInt'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchApproved&value=true&value_type=Bool'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
```

Expected shapes:

```
{"success":true,"name":"GemStoneRsBridgeRoot","oop":1234,"identityId":1}
{"success":true,"root":"GemStoneRsBridgeRoot","keys":
[{"printString":"BookingDraft"}]}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","keyType":"String"
,"depth":4,"value":
{"oop":5678,"classOop":9012,"printString":"aDictionary(...)"},"bridgeValue":
{"type":"dictionary","entries":{"name":{"type":"string","value":"Tariq"}}}}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","shape":
{"totalNodes":6,"nodes":
[{"path":"value.customer.#name","kind":"string"}],"identityGroups":
[{"identityId":1,"oop":1234,"paths":["value.items[2]","value.items[3]"]}]}}
{"success":true,"config":"mapped = BookingDraft ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"BookingDraft","mapped":"BookingD
raft","config":"# Generated by gemstone-rs from a live BridgeValue ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"WorkbenchDraft","keyType":"Strin
g","valueType":"String","oop":3456}
```

Product Direction

The explorer should remain a separate binary crate. The core `gemstone-rs` library stays focused on safe GemStone API access, while the explorer proves higher-level browsing and codegen workflows.

Good next steps:

- keep Marketplace/GitHub screenshots current after UI changes
- run the explorer endpoint smoke checks and VS Code smoke checks after

  touching the embedded webview

- refine workflow feedback around writes, generated-file saves, and live

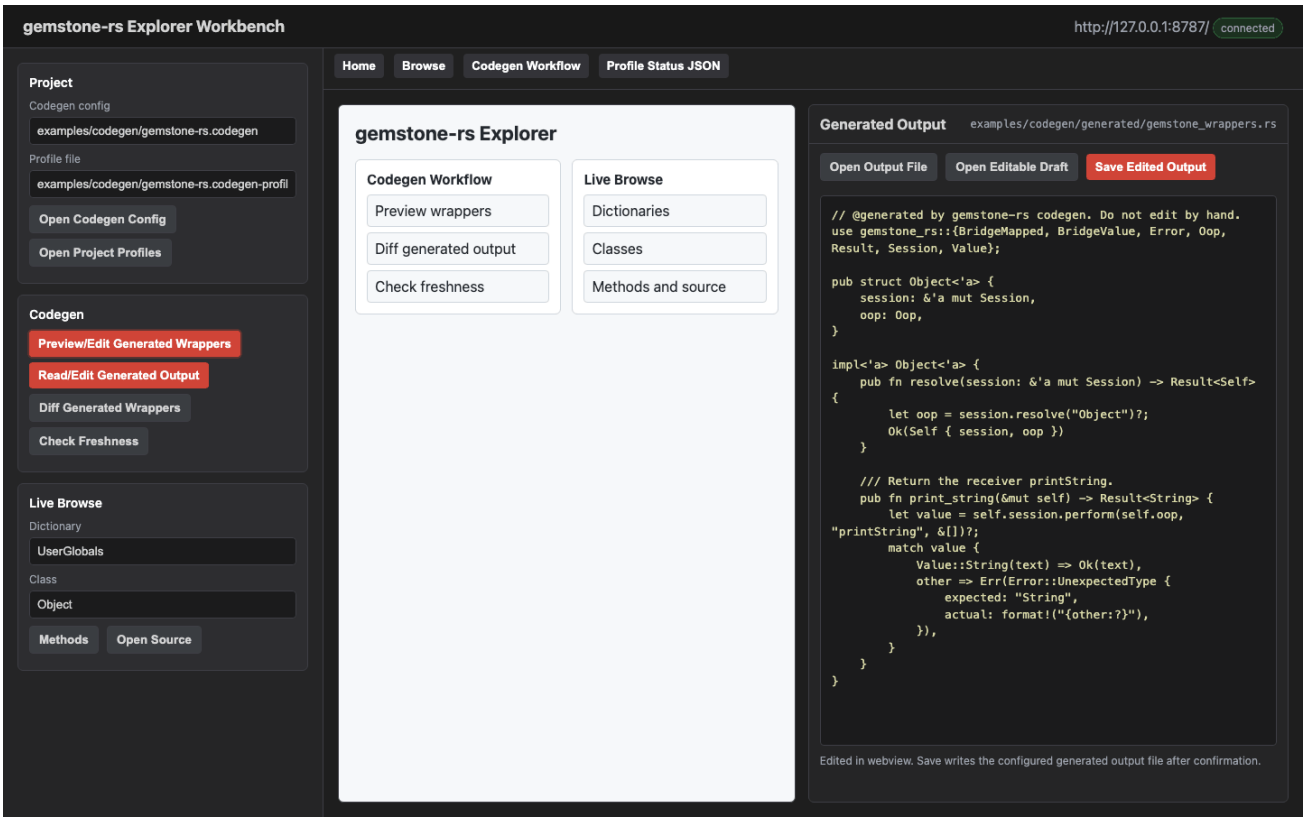
  BridgeRoot probes as real projects expose rough edges

VS Code Workbench

`gemstone-rs Workbench` is the VS Code companion for the Rust CLI and local explorer. It keeps the CLI as the stable contract and adds a sidebar, output panel, generated-file previews, and codegen diff prompts.

Marketplace:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```



Setup

For installed CLI tools:

```
{
  "gemstoneRs.cliPath": "gemstone-rs",
  "gemstoneRs.explorerPath": "gemstone-rs-explorer",
```

```
  "gemstoneRs.codegenConfig": "gemstone-rs.codegen",
  "gemstoneRs.bridgeRoot": "GemStoneRsBridgeRoot"
}
```

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen",
  "gemstoneRs.bridgeRoot": "GemStoneRsBridgeRoot"
}
```

Use the same `GS_*` environment as the CLI.

Sidebar Tree

Open the GemStone RS activity bar item.

The sidebar contains:

- `Dictionaries`
- `Codegen Config`
- `Explorer`

`Dictionaries` expands live dictionaries, classes, protocols, and methods. Selecting a method opens its source in an untitled Smalltalk editor.

`Codegen Config` exposes:

- Discover from Live Stone
- BridgeRoot name from `gemstoneRs.bridgeRoot`
- Generate Mapping Config
- Preview BridgeRoot
- List BridgeRoot Keys
- Put BridgeRoot String
- Put BridgeRoot Symbol
- Put BridgeRoot SmallInt
- Put BridgeRoot Bool
- Remove BridgeRoot Key
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles

- Save Project Profiles
- Export Codegen Profile
- Show Sample Project Profiles
- Create Project Profiles
- Validate Project Profiles
- List Project Profiles
- Show Project Profile
- Resolve Project Profile
- Check Project Profiles
- Open Codegen Docs

`Explorer` exposes:

- Doctor
- Verify Setup
- Verify Live Setup
- Eval Smalltalk
- Show Example Commands
- Generate Explorer Auth Token
- Clear Explorer Auth Token
- Launch Explorer
- Open Explorer Webview

`Comparison` exposes:

- Compare with gemstone-py
- Show All Comparison Status

Command Palette

Commands:

- `GemStone RS: Verify Setup`
- `GemStone RS: Verify Live Setup`
- `GemStone RS: Verify Strict Setup`
- `GemStone RS: Doctor`
- `GemStone RS: Show Environment Template`
- `GemStone RS: Copy Environment Template`
- `GemStone RS: Write .env.gemstone-rs`
- `GemStone RS: Eval Smalltalk`
- `GemStone RS: Browse Dictionaries`
- `GemStone RS: Browse Classes`
- `GemStone RS: Show Example Commands`
- `GemStone RS: Codegen Init`
- `GemStone RS: Codegen Discover`
- `GemStone RS: Generate Mapping Config`

- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`
- `GemStone RS: Codegen Preview`
- `GemStone RS: Codegen Diff`
- `GemStone RS: Codegen Check`
- `GemStone RS: Codegen Explain`
- `GemStone RS: Codegen Generate`
- `GemStone RS: Codegen Preview Profile`
- `GemStone RS: Codegen Diff Profile`
- `GemStone RS: Codegen Check Profile`
- `GemStone RS: Codegen Explain Profile`
- `GemStone RS: Codegen Generate Profile`
- `GemStone RS: Load Project Profiles`
- `GemStone RS: Save Project Profiles`
- `GemStone RS: Export Codegen Profile`
- `GemStone RS: Show Sample Project Profiles`
- `GemStone RS: Create Project Profiles`
- `GemStone RS: Validate Project Profiles`
- `GemStone RS: List Project Profiles`
- `GemStone RS: Show Project Profile`
- `GemStone RS: Resolve Project Profile`
- `GemStone RS: Check Project Profiles`
- `GemStone RS: Generate Explorer Auth Token`
- `GemStone RS: Clear Explorer Auth Token`
- `GemStone RS: Launch Explorer`
- `GemStone RS: Open Explorer Webview`
- `GemStone RS: Open Method Source`
- `GemStone RS: Open Codegen Docs`
- `GemStone RS: Validate py-native Contract`
- `GemStone RS: Validate py-native Samples Fixture`
- `GemStone RS: Validate py-native Smoke Fixture`
- `GemStone RS: Run py-native Smoke`
- `GemStone RS: Show py-native Migration Plan`
- `GemStone RS: Validate py-native Conformance Fixture`
- `GemStone RS: Validate py-native Handoff Bundle`
- `GemStone RS: Show py-native Handoff Bundle`
- `GemStone RS: Validate py-native Publish Receipt`
- `GemStone RS: Show py-native Publish Receipt`
- `GemStone RS: Validate py-native Shared Core Gate`

- `GemStone RS: Compare with gemstone-py`
- `GemStone RS: Show All Comparison Status`

`GemStone RS: Verify Setup` and `GemStone RS: Doctor` both run the CLI `gemstone-rs doctor` report, so the workbench setup view and terminal setup use the same diagnostic path. `GemStone RS: Verify Live Setup` runs `gemstone-rs doctor --live` when credentials and a reachable stone are available. `GemStone RS: Verify Strict Setup` runs `gemstone-rs doctor --strict` for CI-style validation of explicit stone and GCI library settings. All setup checks add the active checkout, CLI, explorer, codegen config, and profile paths, plus whether the configured `gemstoneRs.envFile` exists. When that file exists, Workbench passes `--env-file` automatically to CLI commands and explorer startup. The report also includes the configured BridgeRoot name used by BridgeRoot commands and explorer probes. The checks offer result actions to copy the full report, copy a safe `GS_*` environment export script with a placeholder password, or open the extension settings.

`GemStone RS: Run Setup Assistant` calls the running local explorer at `/api/setup/assistant` and renders the structured env-file, GemStone configuration, GCI library, codegen config, and project profile checks in the output panel. Start the explorer first with `GemStone RS: Launch Explorer`.

`GemStone RS: Show Environment Template`, `GemStone RS: Copy Environment Template`, and `GemStone RS: Write .env.gemstone-rs` call the CLI `gemstone-rs env sample/write` commands. The template uses current non-secret values when available and keeps passwords as placeholders.

`GemStone RS: Show Example Commands` calls `gemstone-rs examples list --json`, renders the installed CLI example index, and lets you run a selected Cargo example in a terminal, copy its command, or open the examples guide. This closes one gemstone-py parity gap: gemstone-rs now has a CLI and editor example launcher instead of only Markdown tables. The matching terminal command is also available as `gemstone-rs examples run <name>` from a source checkout.

Codegen Workflow

1. Set `gemstoneRs.codegenConfig`.
2. Run `GemStone RS: Codegen Discover` or edit the config manually.
3. Run `GemStone RS: Codegen Preview`.
4. Run `GemStone RS: Codegen Diff`.
5. Run `GemStone RS: Codegen Explain`.
6. Run `GemStone RS: Open Codegen Config` or `GemStone RS: Open Project Profiles` when you want to edit configured files in VS Code.
7. Run `GemStone RS: Validate py-native Contract` when you want to confirm the checked-in Rust adapter contract for `gemstone-py-native`.
8. Run `GemStone RS: Validate py-native Samples Fixture` when you want to

confirm the checked-in value/error payload examples for wrapper tests.

9. Run `GemStone RS: Validate py-native Smoke Fixture` when you want to confirm the checked-in dry-run smoke report for adapter consumers.
10. Run `GemStone RS: Run py-native Smoke` when you want adapter smoke checks from VS Code.
11. Run `GemStone RS: Show py-native Migration Plan` when you want the `gemstone-py-native` shared-core checklist and publish-verification status.
12. Run `GemStone RS: Validate py-native Conformance Fixture` when you want the PyO3 module/session/shim surface checked against the Rust core.
13. Run `GemStone RS: Validate py-native Handoff Bundle` or `GemStone RS: Show py-native Handoff Bundle` when you want the downstream wrapper manifest and acceptance criteria.
14. Run `GemStone RS: Validate py-native Publish Receipt` or `GemStone RS: Show py-native Publish Receipt` when you want verified TestPyPI/PyPI workflow run ids, install commands, and package checks in the output panel.
15. Run `GemStone RS: Validate py-native Shared Core Gate` when you want all checked-in py-native fixtures validated by one downstream CI gate.
16. Run `GemStone RS: Open Generated Output`.
17. Run `GemStone RS: Codegen Generate`.

`Codegen Generate` runs the diff first. If output would change, it opens the diff and asks before writing.

`Codegen Explain` runs `codegen explain --json` and renders the output path, generated test stubs, wrapper classes, selectors, return types, and BridgeRoot mappings in the output panel. Result actions can copy the summary, copy the raw JSON, open the JSON in an editor, or open the config file. `Open Codegen Config` and `Open Project Profiles` open the configured files directly from settings, which is useful from the webview and command palette. `Open Generated Output` uses the same structured explain output to open the current generated wrapper file directly.
`Validate py-native Contract` runs `gemstone-rs py-native check --json` against `gemstoneRs.pyNativeFixture`, then renders the path, status, and contract version in the output panel with actions to copy the report or open the fixture.
`Validate py-native Samples Fixture` runs `gemstone-rs py-native check-samples --json` against `gemstoneRs.pyNativeSamplesFixture`, then renders the fixture freshness, value count, and error count. `Validate py-native Smoke Fixture` runs `gemstone-rs py-native`

`check-smoke --json` against `gemstoneRs.pyNativeSmokeFixture`, then renders the same fixture freshness view for the dry-run adapter smoke report. `Run py-native Smoke` runs `gemstone-rs py-native smoke --json`, prompts for dry-run or live mode, and shows every adapter step with copyable output. `Show py-native Migration Plan` runs `gemstone-rs py-native migration --json` and renders the current `gemstone-py-native` wrapper migration checklist. `Validate py-native Conformance Fixture` runs `gemstone-rs py-native check-conformance --json` against `gemstoneRs.pyNativeConformanceFixture`, then renders module function, session method, compatibility method, fixture, and scaffold-file counts. `Validate py-native Handoff Bundle` runs `gemstone-rs py-native check-handoff --json` against `gemstoneRs.pyNativeHandoffFixture`, and `Show py-native Handoff Bundle` runs `gemstone-rs py-native handoff --json` to render the artifacts, schemas, commands, checks, and downstream acceptance criteria. `Validate py-native Publish Receipt` runs `gemstone-rs py-native check-publish-receipt --json` against `gemstoneRs.pyNativePublishReceiptFixture`, and `Show py-native Publish Receipt` runs `gemstone-rs py-native publish-receipt --json` to render TestPyPI/PyPI workflow run ids, install commands, verified timestamps, and package checks.
`Validate py-native Shared Core Gate` runs `gemstone-rs py-native check-all --json`, then renders capabilities, samples, smoke, compatibility, conformance, handoff, and publish-receipt fixture status as one copyable report for downstream `gemstone-py-native` CI.

Inside `GemStone RS: Open Explorer Webview`, the Codegen buttons use the same config/profile settings but keep the review loop in one pane. `Preview/Edit Generated Wrappers`, `Read/Edit Generated Output`, `Preview/Edit Profile`, and `Read/Edit Profile Output` render generated Rust source in an editable webview textarea. From there you can:

- open the configured generated output file in a normal VS Code editor
- open the current webview text as an untitled editable draft
- save the edited text back to the generated output file after a confirmation

The save handler refuses paths outside `gemstoneRs.checkoutPath` or the active workspace, so an accidental config path cannot overwrite files elsewhere on the machine.

When a project profile file is checked in, use the profile variants instead: `Codegen Preview Profile`, `Codegen Diff Profile`, `Codegen Check Profile`, `Codegen Explain Profile`, and `Codegen Generate Profile`. They prompt for a profile name and `gemstone-rs.codegen-profiles.json`, resolve the profile's config/root fields, and then run the same codegen operation.

Object Mapping Workflow

The workbench follows the same mapping ladder as the Rust API:

Task	Use
Inspect a live value	BridgeRoot Value and BridgeRoot Shape

Task	Use
Turn a shape into config	BridgeRoot Mapping Preview or Generate Mapping Config
Keep generated code reviewable	Codegen Preview, Diff, Check, and Generate
Work with an existing OOP	`Remote<T>` in Rust code, with explicit `refresh` and `save`

This is not transparent persistence. The extension should help users inspect, preview, diff, and generate mapping code, but it should not hide writes or pretend GemStone field access is local Rust field access.

Use `GemStone RS: Generate Mapping Config` when you want the live stone to inspect a GemStone class and propose a `BridgeMapped` config. The command asks for:

- config path
- Rust struct name
- GemStone class reference, such as `Object` or `UserGlobals:OkzBooking`

Use `GemStone RS: Preview BridgeRoot` after starting the local explorer. It opens:

```
http://127.0.0.1:8787/api/bridge/root?root=GemStoneRsBridgeRoot
```

Use the embedded explorer's `BridgeRoot Value` action when you want the richer view. The response includes the raw OOP/class/printString summary and a nested `BridgeValue` tree, so dictionaries, arrays, symbol values, and depth-limited object references are visible before you settle on a generated mapping. The `BridgeRoot Keys` result includes `Use` buttons that copy a live key into the inspector fields, and the `Bridge key type`, `Bridge depth`, and `Mapped name` inputs are reused by the value, shape, and mapping-preview probes. Use `BridgeRoot Shape` to turn that tree into relationship paths, node kinds, key policy, nil counts, opaque OOP counts, report-local identity ids, and repeated object references. The webview renders both the per-path table and a separate repeated-identity table when the same GemStone object is seen from multiple paths. Use `BridgeRoot Mapping Preview` in that same panel to infer a starter `BridgeMapped` codegen config from the selected live value before saving it to the project.

Use the generated config as a starting point. Review dictionary key policy, opaque OOP fields, optional fields, arrays, repeated identities, and relationship paths before generating wrappers or mapping structs.

Use `GemStone RS: List BridgeRoot Keys` for the same inspection path without opening a browser; it runs `gemstone-rs bridge keys --root <bridge-root>` and writes the key OOPs, class OOPs, `printString` values, and identity ids to the output panel.

Set `gemstoneRs.bridgeRoot` when a project uses a custom bridge dictionary global. The workbench passes that value to CLI commands as `--root` and to the explorer API as the `root=` query parameter.

Use `GemStone RS: Put BridgeRoot String` when you want a quick live-write smoke test from VS Code. It asks for a key, whether that key is a String or Symbol, and a string value, then runs:

```
gemstone-rs bridge put-string <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Put BridgeRoot Symbol` for the same workflow when the stored value should be a GemStone Symbol:

```
gemstone-rs bridge put-symbol <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Put BridgeRoot SmallInt` or `GemStone RS: Put BridgeRoot Bool` when the stored value should be a GemStone SmallInteger or Boolean:

```
gemstone-rs bridge put-smallint <key> <value> --key-type String --root
GemStoneRsBridgeRoot
gemstone-rs bridge put-bool <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Remove BridgeRoot Key` to remove one BridgeRoot key after a String/Symbol key-type prompt and confirmation prompt:

```
gemstone-rs bridge remove <key> --key-type String --root GemStoneRsBridgeRoot
```

Use `GemStone RS: Run Generated Mapping Example` to run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Embedded Explorer Webview

Use `GemStone RS: Launch Explorer` first. It starts `gemstone-rs-explorer` in a terminal and opens the browser. Then use `GemStone RS: Open Explorer Webview` to embed the local explorer inside VS Code.

The webview points at:

```
http://127.0.0.1:8787/
```

If you set `gemstoneRs.explorerAuthToken`, `GemStone RS: Launch Explorer` starts the server with `--auth-token-env GEMSTONE_RS_EXPLORER_TOKEN` so the token is not printed in the terminal command. Browser and webview URLs include the matching `token=` query parameter. Use `GemStone RS: Generate Explorer Auth Token` to create a local random token, save it to VS Code settings, and copy it to the clipboard. Use `GemStone RS: Clear Explorer Auth Token` to return to the default loopback-only, no-token mode.

It is still the same loopback-only explorer process. If the explorer is not running, the webview will show a connection error and the output panel will show the URL to start.

The embedded page now acts as the main IDE surface for the local explorer. The iframe remains the full browser UI, while the side inspector renders structured setup checks, live dictionary/class/protocol/method browsing, method source previews, project profile freshness tables with Preview/Diff/Check/Generate buttons, Codegen explain summaries, generated source, colorized diffs, BridgeRoot key pickers, nested `BridgeValue` trees, shape reports with repeated identity groups, mapping previews, and comparison status cards. The same shell can hand off to native VS Code commands for generated wrapper preview, diff, check, opening config/profile/generated files, editing generated source in the webview, saving edited generated output with confirmation, generate-with-confirmation, profile checks, docs, and opening the last generated output file reported by the explorer. If the explorer is not running, the webview prompts with Launch Explorer, Open Browser, and Copy URL fallback actions.

The webview also exposes comparison status commands. `Compare with gemstone-py` runs `gemstone-rs compare gemstone-py --status --json` and renders the answer, parity score, remaining batch count, next batch when one exists, top gap, and follow-up CLI commands in the GemStone RS output panel. `Show All Comparison Status` renders the aggregate status for every comparison target, including the combined batch/hour estimate.

The explorer page can load and save the selected codegen config file, posting the editor contents as the request body; saves still require the explorer to run with `--allow-write`. Use `Refresh Configs` in the embedded page to populate the `Known configs` picker from the explorer process working tree, or set `Config root` in the page when the explorer was launched from a different directory. The page also keeps a local `Recent configs` picker for paths used by Load, Save, Preview, Diff, Check, and Generate. Use `Save Profile` when you want to switch between named codegen workflows; a profile stores the config root, config path, mapped Rust type, and GemStone class in local browser storage. `Export Profile` writes a JSON payload into `Profile JSON`, and `Import Profile` merges pasted profile JSON from another browser or teammate. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the configured codegen root, and `Save Project Profiles` writes the saved profile list back when the explorer was launched with `--allow-write`. Project profile payloads are schema-validated before writing, and imports report which profiles are new, replaced, or unchanged. It remembers its current fields locally, so repeated codegen or BridgeRoot checks keep the same config path and class selection.

Use `GemStone RS: Show Sample Project Profiles` to open the built-in sample from `gemstone-rs profile sample`, `GemStone RS: Create Project Profiles` to write that sample into a project file with `gemstone-rs profile init`, and `GemStone RS: Validate Project Profiles` when you want a direct CLI check without opening the explorer. Validation

runs `gemstone-rs profile validate` against the selected profile file and writes the result to the GemStone RS output panel. `GemStone RS: List Project Profiles` and `GemStone RS: Show Project Profile` run `gemstone-rs profile list/show` and render the parsed config/root/mapped/class fields in the same output panel. `GemStone RS: Resolve Project Profile` runs `gemstone-rs profile resolve` and shows the config path that profile-driven codegen will use. `GemStone RS: Check Project Profiles` runs `gemstone-rs profile check --json` across every profile and renders an aggregate summary with ok, stale, and error counts before you commit. The result message can copy the report or open the profile file for quick repair. Profile-specific commands use a QuickPick populated from the project profile file. The extension also contributes JSON validation for files named `gemstone-rs.codegen-profiles.json` and profile check reports named `gemstone-rs.profile-check.json`.

Develop the Extension

```
cd vscode-gemstone-rs-workbench
npm ci
npm run check
npm run test:smoke
npm run package -- --out gemstone-rs-workbench-0.3.4.vsix
```

From the repository root:

```
make vscode-package
```

Later Feature Work

The embedded webview now covers the main Codegen, profile, setup, comparison, generated-output editing, BridgeRoot inspection, live browsing, source preview, open-file loops, nested BridgeValue rendering, and committed Marketplace/GitHub visual assets. Shape reports now also show repeated OOP identity groups, so users can see every relationship path pointing at the same GemStone object. Next polish should keep Marketplace/GitHub screenshots current, run explorer/API smoke checks after UI changes, and refine workflow feedback as real projects expose rough edges.

Screenshot Workflow

The repository keeps screenshots in `docs/assets/` so the README, guides, PDFs, Marketplace text, and articles can point at stable image paths.

Explorer Screenshot

`docs/assets/explorer-home.png` is captured from the local explorer at a 1440x1500 viewport.

Install Playwright once:

```
python3 -m pip install playwright
python3 -m playwright install chromium
```

If Playwright is not installed, the capture script falls back to a local Chrome, Chromium, or Microsoft Edge executable when one is available.

Then refresh the screenshot:

```
make screenshots
```

The target writes:

```
docs/assets/explorer-home.png
docs/assets/workbench-codegen-edit-flow.png
docs/assets/workbench-codegen-edit-flow.gif
```

The target starts:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

It waits for:

```
http://127.0.0.1:8787/health
```

Then it writes:

```
docs/assets/explorer-home.png
```

If the explorer is already running, capture it without starting another server:

```
python3 scripts/capture_explorer_screenshots.py --no-server --url http://
127.0.0.1:8787/
```

Run a dry path check without launching a browser:

```
python3 scripts/capture_explorer_screenshots.py --check
```

Validate all committed visual assets and Markdown image references:

```
make visual-asset-check
```

That check verifies the VSIX icon path, required Marketplace/GitHub assets, local Markdown image links, and repository `raw.githubusercontent.com` image links that point back into this project.

Validate local Markdown links and same-repository raw GitHub links:

```
make docs-link-check
make docs-index-check
```

That check is offline; it does not probe public websites, but it does catch missing local files, missing directories, broken Markdown heading anchors, and main guides that are missing from `README.md` or `docs/README.md`.

The screenshot path does not require GemStone credentials because the explorer home page renders before live endpoints are called. Live browse/codegen data still requires the normal `GS_*` environment.

Workbench Marketplace Images

When the VS Code webview changes, refresh `docs/assets/explorer-home.png`, `docs/assets/workbench-codegen-edit-flow.png`, and `docs/assets/workbench-codegen-edit-flow.gif`, then capture any additional Marketplace/GitHub GIFs from VS Code with the same visible flow:

1. Run `GemStone RS: Launch Explorer`.
2. Run `GemStone RS: Open Explorer Webview`.
3. Click `Preview/Edit Generated Wrappers`.
4. Show the generated-source editor with `Open Output File`, `Open Editable Draft`, and `Save Edited Output`.
5. Capture a still screenshot or short GIF for the Marketplace listing.

The committed still image and GIF show the embedded explorer, the Codegen panel, and the editable generated-output pane so users can see the review/edit/save loop without reading the command list. Capture an additional manual GIF only when you need to show the real VS Code confirmation prompt.

After Capture

Regenerate PDFs after refreshing screenshots:

```
python3 docs/build_pdf_docs.py
python3 docs/build_pdf_docs.py --check
```

For a release pass, run:

```
make visual-asset-check
make docs-link-check
make docs-index-check
make verify
make vscode-package
```

Performance and Safety

`gemstone-rs` keeps the low-level GemStone/S GCI boundary explicit. The goal is to make Rust services and tooling fast without hiding session, transaction, or threading rules.

Dynamic GCI Loading

The GCI library is loaded at runtime. Configure it with one of:

```
export GS_LIB=/path/to/GemStone64Bit/lib
export GS_LIB_PATH=/path/to/GemStone64Bit/lib
export GEMSTONE=/path/to/GemStone64Bit
```

The loader already reports which source selected `libgcirpc` in `gemstone-rs doctor`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. It should keep improving in these directions:

- detect architecture mismatches early
- avoid requiring GemStone headers at Rust build time
- keep all unsafe C ABI calls isolated in `gemstone-gci`

Session Threading

`Session` is deliberately non-`Send` and non-`Sync`. Treat a session as owned by the thread that logged it in.

For web servers:

- use `SessionWorker` when you want one reusable dedicated session lane
- use `SessionWorkerPool` when a service needs a bounded set of reusable lanes
- use `spawn_blocking` or framework blocking helpers for simple per-request probes
- prefer a small `SessionWorkerPool` for web services and reuse

  `gemstone-rs-axum` or `gemstone-rs-actix` instead of copying handler code

- use the adapter `*_from_env` helpers when the HTTP process should start

  before GemStone credentials are configured; `/health/gemstone` will report unavailable until the pool is ready

- keep `x-gemstone-rs-adapter`, `x-gemstone-rs-route`,

  `x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path` headers enabled in local services so route smoke tests and proxy logs can identify health paths and correlate requests

- keep `x-gemstone-rs-request-lifecycle` and

  `x-gemstone-rs-request-duration-us` enabled to verify the packaged handler lifecycle and measure route-handler time without custom middleware

- keep framework middleware smoke markers such as

  `x-gemstone-rs-example-middleware` in checked examples so adapter upgrades do not bypass application middleware

- run `python3 scripts/framework_route_smoke.py --live` or

  `make framework-live-smoke` in a credentialed environment before release when web adapters must prove `/health/gemstone` returns `{"result":7}`

- keep transaction boundaries explicit
- do not share a live session between async tasks

Benchmark Smoke Test

Use the lightweight benchmark smoke script for a coarse local sanity check:

```
ITERATIONS=20 scripts/benchmark_smoke.sh "3 + 4"
```

This measures CLI startup, login, eval, and logout together. It is not a microbenchmark. A future benchmark suite should separate:

- GCI library load time

- login/logout time
- eval/perform latency
- string and array marshalling
- BridgeRoot read/write latency
- `BridgeValue` materialization depth and shape-report latency
- `Remote<T>` refresh/save latency for dictionary-backed mapped objects
- codegen wrapper overhead

Rust vs Python

Compare `gemstone-rs` and `gemstone-py` only with equivalent workloads:

- same stone
- same host
- same GemStone user
- same expression or selector
- warm and cold runs reported separately

The first useful comparison is:

```
Rust:   cargo run -p gemstone-rs-cli -- eval "3 + 4"
Python: python -m gemstone_py.cli eval "3 + 4"
```

After that, compare generated-wrapper calls and BridgeRoot mapping operations. For mapping comparisons, report the materialization profile (`shallow` or `deep(n)`), dictionary key policy, array policy, and whether repeated opaque OOPs were present in the shape report.

Safety Checklist

- Keep `Session` non-`Send` and non-`Sync`.
- Keep `unsafe` in `gemstone-gci`.
- Prefer typed `Value` / `Oop` conversions over raw integer handling.
- Commit or abort explicitly after writes.
- Keep object mapping explicit: no hidden GemStone calls from Rust field access

  and no save-on-drop behavior.

- Bind local tools to `127.0.0.1` by default.
- Keep explorer eval/write operations opt-in.

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgcirpc loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.
  Also exposes gemstone_rs::py_native as a narrow adapter contract.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The Rust side now exposes `gemstone_rs::py_native`, which is deliberately plain Rust and dependency-free. A PyO3 crate can wrap these types without depending on internal `Session` details:

- `PyNativeConfig`
- `PyNativeConfigSummary`
- `PyNativeValue`
- `PyNativeErrorInfo`
- `PyNativeSession`
- `capabilities()`

Print the adapter contract from the CLI when a wrapper build, CI job, or documentation generator needs a stable machine-readable surface:

```
gemstone-rs py-native capabilities
gemstone-rs py-native capabilities --json
```

The JSON output is covered by `schemas/gemstone-rs.py-native.schema.json` and the smoke output is covered by `schemas/gemstone-rs.py-native-smoke.schema.json`. Value/error samples are covered by `schemas/gemstone-rs.py-native-samples.schema.json`. The migration checklist output is covered by `schemas/gemstone-rs.py-native-migration.schema.json`. The Python compatibility shim output is covered by `schemas/gemstone-rs.py-native-compat.schema.json`. The wrapper conformance output is covered by `schemas/gemstone-rs.py-native-conformance.schema.json`. The downstream handoff bundle output is covered by `schemas/gemstone-rs.py-native-handoff.schema.json`. The publish receipt output is covered by `schemas/gemstone-rs.py-native-publish-receipt.schema.json`. The one-command downstream gate output is covered by `schemas/gemstone-rs.py-native-check-all.schema.json`. The VS Code workbench packages these schemas for editor validation. A checked-in fixture lives at `examples/py-native/gemstone-rs.py-native.json`, the value/error sample fixture lives at `examples/py-native/gemstone-rs.py-native-samples.json`, and the dry-run smoke fixture lives at `examples/py-native/gemstone-rs.py-native-smoke.json`, with the compatibility fixture at `examples/py-native/gemstone-rs.py-native-compat.json`, and the conformance fixture at `examples/py-native/gemstone-rs.py-native-conformance.json`, with the final handoff bundle at `examples/py-native/gemstone-rs.py-native-handoff.json`, and the native wheel publish receipt at `examples/py-native/gemstone-rs.py-native-publish-receipt.json`, so downstream wrapper CI can diff the contract without requiring a live stone. Use the CLI check in CI:

```
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json --json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json --json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
--json
gemstone-rs py-native smoke --dry-run
gemstone-rs py-native smoke --dry-run --json
gemstone-rs py-native migration
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility
gemstone-rs py-native compatibility --json
gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-
compat.json
gemstone-rs py-native conformance
gemstone-rs py-native conformance --json
gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-
conformance.json
gemstone-rs py-native handoff
gemstone-rs py-native handoff --json
gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-
handoff.json
gemstone-rs py-native publish-receipt
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-publish-receipt examples/py-native/gemstone-rs.py-native-
publish-receipt.json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
```

`py-native migration --json` is intentionally not a replacement for doing the Python-side work. It is the shared Rust-side checklist that CLI, CI, docs, and the VS Code workbench can render consistently while `gemstone-py-native` keeps its additive `RustCoreSession` bridge over `gemstone_rs::py_native` green through live smoke and published-wheel verification. `py-native handoff --json` is the single manifest to hand to downstream `gemstone-py-native` work: it lists each artifact, schema, generation command, validation command, and required acceptance check. `py-native publish-receipt --json` records the verified TestPyPI and PyPI workflow runs, install commands, and package checks for the Rust-backed `gemstone-py-native` wheel release. `py-native check-all --json` is the CI acceptance gate for that handoff: it validates the checked-in capabilities, samples, smoke, compatibility, conformance, handoff, and publish-receipt fixtures in one report.

To create a starter PyO3 wrapper crate from the installed CLI:

```
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
cd ./gemstone-py-native-starter
cargo run
python -m venv .venv
```

```
source .venv/bin/activate
python -m pip install maturin pytest
maturin develop
python -c 'import gemstone_py_native; print(gemstone_py_native.migration_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.compatibility_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.conformance_json())'
python -c 'import gemstone_py_native; print(gemstone_py_native.handoff_json())'
python -c 'import gemstone_py_native_compat;
print(gemstone_py_native_compat.compatibility_report()["returnPolicy"])'
pytest
```

The scaffold writes `pyproject.toml`, `src/lib.rs`, `PYTHON.md`, `python/gemstone_py_native_compat.py`, and a Python smoke test. It is deliberately thin: Python calls PyO3 functions/classes, and those delegate into `gemstone_rs::py_native`. The generated crate currently uses PyO3 0.28 for Python 3.14 compatibility and keeps PyO3's `extension-module` flag behind a Cargo feature so `cargo run` works while `maturin develop` still builds a proper Python extension. It exposes `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, and `handoff_json`, so Python wrapper CI can inspect the adapter contract, compatibility method map, conformance target, handoff manifest, and live/publish verification checklist from the generated module. The generated `NativeSession` also maps the Rust session operations for eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions without adding Pythonic return conversion in the native layer. It includes JSON value helpers such as `eval_json` and `perform_json` so Python code can translate the stable `PyNativeValue` shape without reimplementing Rust value classification. The generated compatibility shim demonstrates the package-layer policy: direct object identity becomes `OopHandle`, raw native OOPs stay below the package boundary, typed value-to-OOP helpers are explicit opt-in methods, and value-returning calls become dictionaries decoded from the Rust JSON contract. The CLI exposes the same method map as `py-native compatibility --json`, including every generated Python method, the underlying `NativeSession` method, the native return type, and the Python return type. The generated `conformance_json()` output is the higher-level integration target: extension module functions, raw `NativeSession` methods, compatibility shim methods, fixture paths, and scaffold files.

From a gemstone-rs source checkout, verify that the embedded scaffold still compiles against the local Rust core with:

```
python3 scripts/check_py_native_pyo3_scaffold.py
```

Run the dry-run contract check from a source checkout when you also want to exercise the example binary:

```
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
```

Run it against a live stone:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
cargo run -p gemstone-rs --example python_native_adapter
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `value_to_oop(value)`
- `perform(oop, selector, args)`
- `commit()`
- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

Downstream PyO3 Shape

The downstream `gemstone-py-native` crate should stay thin:

```
use gemstone_rs::py_native::{PyNativeConfig, PyNativeSession};

struct NativeSession {
    inner: PyNativeSession,
}

impl NativeSession {
    fn login(config: PyNativeConfig) -> gemstone_rs::Result<Self> {
        Ok(Self {
            inner: PyNativeSession::login(config)?,
        })
    }

    fn eval(&mut self, source: &str) -> gemstone_rs::Result<String> {
        Ok(format!("{:?}", self.inner.eval(source)?))
    }
}
```

Actual PyO3 code should translate `PyNativeValue` into Python objects and translate `PyNativeErrorInfo` into Python exceptions. Keep `PyNativeSession` unsendable unless a dedicated worker-thread wrapper is used.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Scaffold or adapt the starter PyO3 crate with `gemstone-rs examples scaffold py_native_pyo3_adapter`.
3. Keep `scripts/check_py_native_pyo3_scaffold.py` green while the starter evolves.
4. Keep the existing `gemstone-py-native` `RustCoreSession` bridge delegating to `gemstone_rs::py_native`.
5. Continue reducing duplicated native loading code in `gemstone-py-native`.
6. Run the existing `gemstone-py` native backend and live tests through the Rust-backed native path.
7. Keep pure Python fallback behavior and current sync return behavior backward compatible.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

0.2.1 / Workbench 0.3.1 Notes

- CLI setup: global `--env-file`, `doctor --strict`, and safer `.env.gemstone-rs` template loading across CLI and explorer startup.
- Codegen: `codegen explain --json`, schema files, and a generated-wrapper compile smoke test.
- Mapping: missing keys, nested dictionaries, and array reads now report richer path context such as `booking.items[2]`.
- Explorer: setup assistant, env sample/write, and a structured Codegen

Explain action.

- VS Code: `GemStone RS: Verify Strict Setup` and automatic env-file passing when `gemstoneRs.envFile` exists.

Workbench 0.3.2 Notes

- VS Code: `GemStone RS: Run Setup Assistant` renders the explorer `/api/setup/assistant` checks in the output panel.
- CI: schema copies and `codegen explain --json` output are validated by `scripts/validate_codegen_schemas.js`.
- CI: `compare ... --json`, including status, scorecard, and parity views, is validated against the same structural contract documented in `schemas/gemstone-rs.compare.schema.json`.
- Examples: the generated-wrapper smoke crate now runs `cargo test`.

Workbench 0.3.3 Notes

- CLI: `codegen explain-profile [--json] <profile-name> [profile-file]` resolves project profiles before rendering codegen summaries.
- VS Code: `GemStone RS: Codegen Explain` and `GemStone RS: Codegen Explain Profile` render structured classes, selectors, mappings, and test stubs.

0.2.2 / Workbench 0.3.4 Notes

- Core: project profile check reports now live in `gemstone-rs` and are shared by CLI, explorer, and VS Code.
- Explorer: `/api/codegen/profiles/check` returns the shared JSON report and the browser renders a profile status table with Preview, Diff, Check, and Generate actions.
- Tooling: `schemas/gemstone-rs.profile-check.schema.json` documents the shared report shape, and CI runs a real explorer HTTP endpoint smoke test.
- Tooling: `scripts/version_check.py` now covers release command snippets,

workflow defaults, VSIX filename examples, and generated Axum/Actix scaffold adapter versions.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile list --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile show default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile resolve default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile check --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile sample > /tmp/gemstone-rs.codegen-
profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
diff -u examples/codegen/gemstone-rs.codegen-profiles.json /tmp/gemstone-rs.codegen-
profiles.json
```

- Refresh screenshots when the explorer or workbench UI changed:

```
make screenshots
make visual-asset-check
make docs-link-check
make docs-index-check
```

- Run local verification:

```
python3 scripts/version_check.py
python3 scripts/crate_metadata_check.py
make verify
```

```
make vscode-package
python3 docs/build_pdf_docs.py
```

Or use the dry-run release wrapper:

```
DRY_RUN=1 scripts/release_all.sh 0.2.2
```

`make verify` includes version and crate metadata checks. The version check guards release workflow defaults, command snippets, VSIX filename examples, and generated Axum/Actix scaffold adapter versions. The crate metadata check also confirms the root README layout lists every publishable crate. The same verification pass then checks that PDF generation completes, produces non-empty PDF files, and exactly matches the configured PDF target set. The release wrapper writes repository-relative SHA256 entries and verifies the expected VSIX plus every PDF with `scripts/verify_release_artifacts.py`. `make verify` also runs an offline release-asset verifier smoke test so checksum mismatches and missing downloaded assets stay covered without contacting GitHub. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms. The VSIX filename uses `vscode-gemstone-rs-workbench/package.json`; the crate release tag still uses the workflow `version` input.

Commit Review Pass

Before publishing, make a deliberate release commit review pass:

```
git status --short
git diff --stat
make verify
```

Confirm the release commit includes any regenerated PDFs, refreshed screenshots, profile schema changes, sample profile updates, and VSIX docs. Keep publishing for a separate step unless the release workflow is intentionally being run from that commit.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
  -f version=0.2.2 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`
3. `gemstone-rs`
4. `gemstone-rs-axum`
5. `gemstone-rs-actix`
6. `gemstone-rs-cli`
7. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:

```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Manual end-to-end publishing remains available through the release wrapper:

```
PUBLISH_CRATES=1 PUBLISH_VSIX=1 CREATE_GITHUB_RELEASE=1 VERIFY_PUBLIC=1 DRY_RUN=0
scripts/release_all.sh 0.2.2
```

The GitHub `Release` workflow now calls the same `scripts/release_all.sh` wrapper, so local and Actions releases share the same verification, packaging, checksum, publish, GitHub release, and post-publish verification path.

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.2
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX, `SHA256SUMS`, and every generated PDF
- downloaded GitHub Release assets match the published `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.2
```

The scheduled/manual **Post-Release Verify** workflow runs the same public verification against crates.io, the VS Code Marketplace, and GitHub Release assets. Run it manually when you want to verify a specific version:

```
gh workflow run post-release-verify.yml \
  --ref main \
  -f version=0.2.2 \
  -f verify-github-release=true \
  -f run-live-smoke=false
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, enable `run-live-smoke=true` on the post-release workflow, or run locally:

```
scripts/live_smoke.sh --dry-run
scripts/live_smoke.sh
```

The live smoke coverage includes `doctor --strict --live`, login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, generated wrapper `printString`, the `py-native smoke` shared-core check, the `live_smoke_cookbook` example, the `python_native_adapter` shared-core example, and live Axum/Actix `/health/gemstone` routes.

The script runs the Rust live tests with `--test-threads=1`. GemStone GCI has process-global session state, so live tests should not be run concurrently.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.